

DERELAB: Probing Defeasible Reasoning and Confirmation Bias in LLMs with a Generative Benchmark

Jayanta Sadhu¹, Sayem Shahad², Kenneth Marino¹

¹University of Utah

²Bangladesh University of Engineering and Technology
{jayanta.sadhu, kenneth.marino}@utah.edu
sayem.shahad@buet.ac.bd

Abstract

Defeasible reasoning is a type of reasoning where inferences are drawn from plausible current evidence, but can be retracted upon the introduction of newer evidence. Although recent studies have examined language model behaviors in defeasible reasoning, the datasets have been static and lack a wide coverage of non-monotonic reasoning categories. We introduce DERELAB, a generative framework that produces multi-turn belief-updating conversations from parameterized graph structures across default and inheritance reasoning, with formally verified ground truth at every turn, enabling controlled measurement of how models respond to confirming and disconfirming evidence. This controlled generation process creates a test bed for experimental designs that isolate specific reasoning demands. Applying this capability to the study of confirmation bias, we evaluate seven open and closed large language models and find that all exhibit a systematic tendency to accept congruent evidence while resisting incongruent updates, with several models correctly identifying a weakening update yet failing to revise their conclusion. All our data and code will be made open-source.

1 Introduction

Reasoning is a central capability for any intelligent agent, and it plays a decisive role in performing reliably in real-world applications. The majority of inferences humans make in real life are defeasible in nature (Chater et al., 2011). Unlike controlled environments, practical reasoning is rarely straightforward due to noisy data and incomplete information flow. With the integration of LLMs into many of our day-to-day activities, it has become important to study language model behaviors for defeasible reasoning, which has been relatively understudied till now.

Prior works on defeasible inference have evaluated language models using static datasets through

multiple probing methods. Rudinger et al. (2020) created the first defeasible reasoning natural language dataset; Allaway and McKeown (2025) created a dataset grounded in generics and inheritance properties. These datasets are, however, susceptible to increasing model performance: with the rapid growth of the field, evaluation datasets often become short-lived, and researchers must handcraft new versions or apply targeted modifications. Furthermore, static benchmarks report aggregate accuracy without revealing *why* a model fails or whether failures reflect genuine reasoning deficits rather than surface-level pattern matching.

We address these limitations with DERELAB (**Defeasible Reasoning Lab**), a generative framework that produces multi-turn belief-updating conversations from parameterised graph structures across two canonical non-monotonic reasoning paradigms — default reasoning and inheritance reasoning (Lifschitz, 1989; Horty et al., 1990) — with formally verified ground truth at every turn. DERELAB follows the generative evaluation paradigm of Liu et al. (2025) and extends it to defeasible reasoning, a fundamentally different setting that requires models to retract conclusions when new evidence defeats prior defaults — a capability that relational reasoning benchmarks do not test. Fine-grained configuration parameters over property chaining, branching factor, distractor density, and source-priority structure enables controlled diagnostic experiments at arbitrary difficulty levels without data recollection.

Using DERELAB, we evaluate seven open and closed LLMs and report three findings. *First*, no model achieves strong performance across all conditions: defeasible reasoning is genuinely hard, and accuracy degrades systematically as reasoning complexity increases. *Second*, confirmation bias (Nickerson, 1998) — the tendency to accept evidence that confirms a prior belief while resisting evidence that contradicts it — is present across all

models on default reasoning, revealing that LLMs exhibit a well-documented human cognitive bias when updating beliefs under new evidence. *Third*, several models correctly identify that an update weakens the hypothesis yet still fail to revise their conclusion — suggesting that the failure is not one of understanding but of belief revision itself.

In this paper we: (1) introduce DERELAB, a flexible dataset generation framework for scalable evaluation and cognitive science-inspired probes of LLMs on defeasible reasoning; (2) demonstrate systematic evaluation across default and inheritance reasoning at multiple difficulty tiers, revealing that reasoning complexity and task type jointly determine model failure; and (3) show how the framework enables cognitive science-inspired probes, uncovering universal confirmation bias and a know-but-don’t-output dissociation across current LLMs.

2 Related Work

Defeasible and Non-monotonic Reasoning Preliminary studies on AI and defeasible reasoning focused on formal logics (e.g. Reiter (1978, 1980); Poole (1988)). In recent times, research has been done to construct datasets built upon these principles and evaluate non-monotonic reasoning ability in language models. Xiu et al. (2022) constructed a dataset for non-monotonic proofs and Parmar et al. (2024) used patterns of default reasoning from Lifschitz (1989) to construct QA dataset. Non-monotonic reasoning had also been studied within the context of natural language inference (NLI) (Yanaka et al., 2019b,a; Gubelmann et al., 2023). Although these datasets focused on evaluating non-monotonic reasoning, there was no scope for introducing additional information. Some works (e.g. Zhao et al. (2023); Rudinger et al. (2020)) resolved this problem and were built for defeasible inference. Evaluating the defeasibility of reasoning is a more nuanced way of evaluating non-monotonic reasoning. A formal modal framework for defeasibility in non-monotonic reasoning had been proposed in (Asher and Morreau, 1991). (Allaway et al., 2023) discussed generics and instantiations in logical programming, and (Allaway and McKeown, 2025) created a dataset for defeasible reasoning with the inheritance reasoning category.

Cognitive biases in LLMs The use of Large Language Models (LLMs) in high-stakes decision-making (e.g., Wu et al. (2023); Singhal et al. (2022)) has made it imperative to broaden the study

of biases beyond traditional ethical and social concerns (e.g., Gallegos et al. (2023)). Specifically, there is a growing need to account for cognitive biases and heuristics, which directly impact the rationality of LLMs’ judgments (Hagendorff et al. (2022)). Early research in this area focused on detecting these effects at the level of individual prompts (Talboy and Fuller (2023); Macmillan-Scott and Musolesi (2024)). Other works have investigated the challenge of detection and mitigation, but were limited to particular LLM roles (Pilli (2024); Ye et al. (2025)), or specific domains (Schmidgall et al. (2024); Opedal et al. (2024)). To address the need for large-scale evaluation, follow-up works have proposed comprehensive frameworks, such as those by Echterhoff et al. (2024) and Xie et al. (2024), which aim to systematically benchmark and mitigate these cognitive limitations (e.g., Zhong et al. (2024)).

Sandbox Evaluation Toolkit To take evaluation beyond simple pattern matching, (Clark et al., 2020) introduced the concept of transformers emulating a reasoning algorithm by concluding explicit rules provided in natural language. (Tafford et al., 2021) extended it by enabling iterative 1-hop generation to build a proof reflecting the model’s decisions. Later, (Kiela et al., 2021) and (Thrush et al., 2022) added human annotation in the evaluation loop to address the rapid saturation of static benchmarks. Recognizing the need of human annotation as a bottleneck, (Liu et al., 2025) proposed a framework to generate reasoning datasets given tunable input configurations and seeds.

3 The DERELAB Framework

DERELAB is a generative evaluation framework for defeasible reasoning that overcomes three limitations of existing benchmarks. *First*, unlike static datasets that become obsolete as model capabilities advance (Kiela et al., 2021), DERELAB generates evaluation problems on demand from a scalable configuration space, allowing difficulty to grow with model capability without collecting new data. *Second*, unlike prior defeasible datasets that vary primarily at the semantic level, DERELAB spans a rich distribution of reasoning *structures* — linear default chains, branching inheritance hierarchies, multi-source priority graphs, and quantified uncertainty — covering structural variation that surface-level rephrasing cannot capture. *Third*, fine-grained configuration parameters across both default and in-

heritance reasoning (chain depth, branching factor, exception density, source count, hierarchy depth, sibling structure) make controlled diagnostic ablation a first-class capability. The implementation details are in Appendix A.

3.1 Defeasibility: Scope and Induction

Defeasible reasoning is inherently *non-monotonic*: conclusions warranted by the current information may need to be retracted when new evidence arrives. DERELAB operationalises this directly by presenting information incrementally rather than all at once. Each conversation begins with a partially specified reasoning structure and then reveals additional facts one update at a time. As the premise set evolves, previously supported conclusions may be reinforced, defeated, reinstated, or left unchanged, depending on the structural role of the new information.

This incremental construction induces a dynamic *belief trajectory* over the course of the conversation. An ideal defeasible reasoner should revise its belief state precisely when new evidence changes the underlying inferential structure, while remaining stable under irrelevant or out-of-scope updates.

3.2 Reasoning Types

DERELAB implements the two canonical paradigms of non-monotonic reasoning identified in the formal literature (Lifschitz, 1989), each targeting a structurally distinct mode of defeasible inference.

Default Reasoning. Default reasoning concerns general rules that hold in the absence of specific contradicting information (Reiter, 1980): conclusions are drawn by default and retracted when exceptions arise. In DERELAB, this is instantiated as a parameterised chain of default rules over objects, with updates that introduce confirmations, defeaters, distractors, and source-priority conflicts.

Inheritance Reasoning. Inheritance reasoning concerns the propagation of properties through a taxonomic hierarchy (Lifschitz, 1989; Horty et al., 1990): a property attributed to a class is inherited by its subclasses and instances unless explicitly blocked at some node. In DERELAB, this is instantiated as a configurable taxonomy with both **linear** and **tree** structures, where updates introduce inheritance-blocking exceptions or properties of sibling branches that must be correctly scoped to the hypothesis subject.

3.3 Default Reasoning: Configuration and Question Types

Configuration. We generate default reasoning examples through parameterised graphs controlled by chain length, number of object instantiations, distractor density, and a sparsity factor controlling defeat-edge injection. Two difficulty tiers are defined: **Easy** (shorter chains, fewer objects) and **Hard** (longer chains, more objects and distractors), with full parameter ranges in Appendix A.1. The order in which graph edges are revealed across turns is separately configurable, allowing the belief trajectory to be shaped by design: for instance, placing a defeating update early tests whether a model correctly reinstates a conclusion when subsequent confirming evidence arrives.

Question Types. Drawing on the taxonomy of default reasoning patterns in Lifschitz (1989), we identify three structural question types that cover the principal modes of defeasible inference; Appendix B details how this taxonomy maps onto our graph structures.

- **Type 1—Direct chain manipulation.** Positive or negative assertions about related properties test basic belief updating; exceptions test defeat of default rules.
- **Type 2—Irrelevant information.** An update concerns a property outside the default chain, making this variant irrelevant to the original default rules. This category checks the reasoning ability of isolating irrelevant property information.
- **Type 3—Source priority.** Competing claims from sources of differing reliability test whether models apply priority relations correctly, including when a higher-priority rule bypasses rather than directly overrides a prior defeat.

In Figure 13, a default reasoning example generation procedure is shown in detail.

3.4 Inheritance Reasoning: Configuration and Question Types

Following Lifschitz (1989), we distinguish two structural variants of inheritance reasoning and explicitly considering them as two different categories from here. *Linear inheritance* arranges categories in a single chain from root to entity;

tree inheritance extends this with branching, where multiple sibling subcategories share a parent. The key diagnostic introduced by branching is scope: an update about a sibling branch must not affect the hypothesis subject, which belongs to a different branch entirely.

Configuration. Both variants are parameterised by hierarchy depth; tree inheritance is additionally parameterised by branching factor, controlling how many sibling subcategories share each parent node. Blocking edges are scoped to individual nodes: blocking inheritance at a node severs the chain for that node only, leaving all sibling and cousin nodes unaffected. The model must therefore track whether the hypothesis subject’s specific path to the root carries an active block, rather than whether any path in the graph does. Full parameter ranges appear in Appendix A.1.

Question Types. Unlike default reasoning, inheritance conversations pose only the entailment question (*yes/no/unknown*) after each update. The belief-update question is omitted because the direction of change is directly recoverable from consecutive entailment answers: a transition from *yes* to *no* is by definition a weakening, making a separate effect question redundant. The primary diagnostic is therefore whether the model correctly identifies which updates fall within the scope of the hypothesis subject’s inheritance path and which do not.

In Figure 14 and 15, the generation procedure for two inheritance reasoning examples (linear and tree) generation procedure is shown in detail.

3.5 Graph Resolution

Ground truth labels in DERELAB are produced deterministically by a path-based resolution procedure grounded in the inheritance theory of Horty et al. (1990). Given the accumulated premise set at any conversational turn, the resolver traverses the knowledge graph and returns one of three verdicts for the target hypothesis: ENTAILED, (*yes*), DEFEATED (*no*), or UNDETERMINED (*unknown*). This mechanism produces formally correct, annotation-free labels at every conversational turn when a new information is revealed. The full resolution procedure is described in Appendix A.3.

3.6 Conversation Structure

For all the categories, the conversation body contains: (i) a *pretext section* presenting background

default rules and entity instantiations; (ii) a *hypothesis* stating the entailment question under evaluation; and (iii) a sequence of *update turns*, each delivering one new fact followed by the entailment question. For default reasoning, each update turn is followed by a second *effect question* eliciting whether the update strengthened, weakened, or had no effect on hypothesis support. Default reasoning conversations may chain multiple hypothesis sections via a new hypothesis turn that changes the active subject while preserving the full accumulated premise set. (see Appendix E for details)

3.7 Entity Design and Contamination Prevention

The overlap between benchmark content and pre-training corpora is a recognized threat to the validity of LLM evaluation (Magar and Schwartz, 2022; Al-Lawati et al., 2026; Zhou et al., 2023). Although our proposed framework is flexible to use any type of entity, we present results keeping all entities in the primary DERELAB dataset as *pseudowords* (logatomes) pronounceable but semantically null strings generated programmatically (e.g., *Kitylu*, *Atool*, *Deraw*) (see Appendix D.1 for pseudoword generation details). For defeasible reasoning, a model may produce the correct answer by retrieving a memorised real-world fact (*penguins cannot fly*) rather than by executing the defeasible inference the benchmark intends to measure. Because pseudoword entities carry no prior semantics in any trained model, correct responses can only arise from reasoning over the premises supplied in context.

3.8 Cognitive Bias Probing

A key capability of DERELAB is the ability to embed cognitive science-inspired experiments directly into the evaluation structure. Because the framework controls the content of information updates, it can construct controlled experimental conditions.

As a primary instantiation, we measure *confirmation bias* in defeasible belief updating: the tendency of LLMs to accept evidence congruent with their current belief while resisting contradicting evidence (Nickerson, 1998). Each update turn is assigned a congruence condition based on the model’s own prior predicted answer and the ground truth effect of the update:

Topology	Difficulty Configuration	Conversations	Question Turns
Default reasoning	Easy	150	1,905
Default reasoning	Hard	150	9,325
Linear inheritance	Easy	150	646
Linear inheritance	Hard	150	2,483
Tree inheritance	Easy	150	963
Tree inheritance	Hard	150	3,068
Total	–	900	18,390

Table 1: Canonical DERELAB evaluation set. “Questions” refers to the total number of answer-bearing turns evaluated across all conversations in each split.

Cond.	Prior belief	Ground-truth effect
C1	yes	strengthening
C2	yes	weakening
C3	no	no effect (negative update)
C4	no	no effect (neutral distractor)
C5	unknown	strengthening / weakening

C1 tests congruent updating, whereas **C2** is the primary confirmation-bias condition: the model should revise its belief downward but may resist doing so. **C3** tests whether models respond to surface negativity rather than logical scope, while **C4** serves as a neutral baseline. **C5** captures anchoring under uncertainty, where the model begins uncertain and receives evidence that should resolve the hypothesis in one direction. Confirmation bias is measured as the accuracy gap between C1 and C2 turns within the same conversation, with matched chain structure and formally verified labels providing a clean within-subject experimental design. Full operationalization details appear in Appendix H.

4 Experimental Setup

4.1 Dataset

We evaluate models on three DERELAB reasoning topologies: default reasoning, linear inheritance, and tree inheritance. For each topology, we generate easy and hard splits, with 150 conversations per split. This gives 900 canonical evaluation conversations in total. All canonical examples use pseudoword entities to avoid contamination from memorized real-world facts. Table 1 shows this in further detail. The easy and hard splits differ in graph size, density, and reasoning depth. Easy examples use shorter chains, fewer objects, and fewer distractor attributes; hard examples use longer chains, more objects, and denser defeasible structures.

We additionally construct a robustness dataset:

Model	Type
gpt-5.1	Closed, standard
gpt-5-mini	Closed, compact reasoning
Gemma-4-31B-it	Open, instruction-tuned
Gemma-4-E4B	Open, instruction-tuned
Llama-3.1-70B	Open, instruction-tuned
Llama-3.1-8B	Open, instruction-tuned
Qwen3-32B	Open, thinking

Table 2: Models evaluated in our experiments.

a reseeded set of 1,200 examples to test whether results are stable across random seeds. This set contains 20 seeds, 2 topologies, and 30 conversations per topology per seed.

4.2 Models

We evaluate both closed and open-weight models, covering different model families, sizes, and inference styles. The closed GPT models provide a strong frontier-model reference point. The Llama and Gemma models provide open-weight comparisons across model families and scales. Qwen3-32B is included for comparison as a reasoning model (Table 2). (see Appendix K for more details)

5 Results and Discussions

The following sections detail the major findings of our experiments.

5.1 Overall Reasoning Accuracy

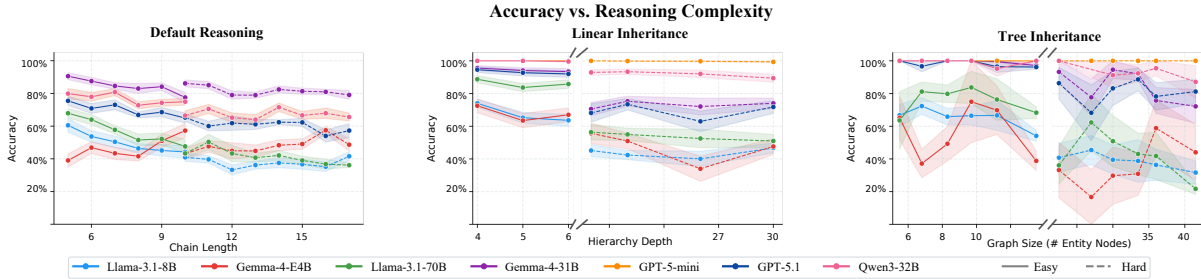
How capable are current LLMs at defeasible reasoning, and does capability vary systematically across reasoning type and task difficulty?

We evaluate all the models on the generated dataset across default and inheritance reasoning at easy and hard difficulty tiers. We report the turn-level accuracy against ground truth labels and performance with respect to reasoning complexity in Figure 1.

We notice a significant performance gap between the abilities of reasoning models and instruction models, where reasoning models clearly outperform the latter. Within task type, default reasoning is consistently harder than inheritance reasoning, and the belief-update task (strengthening / weakening / no effect) is the most demanding of all conditions. Larger models perform better within each family: Gemma-4-31B outperforms Gemma-4-E4B and Llama-3.1-70B outperforms Llama-3.1-

	Default Reasoning				Linear Inheritance		Tree Inheritance		
	Entail Easy	Entail Hard	Belief Update Easy	Belief Update Hard	Easy	Hard	Easy	Hard	Avg
GPT-5-mini	70.6	62.3	71.5	65.4	100.0	99.7	100.0	99.9	83.7
Gemma-4-31B	90.6	90.6	64.7	61.4	94.3	73.7	98.3	85.7	82.4
Qwen3-32B	68.7	60.3	68.5	63.4	99.9	90.6	99.8	94.1	80.7
GPT-5.1	64.3	55.1	64.2	56.2	93.0	70.5	96.6	80.3	72.5
Llama-3.1-70B	50.8	31.4	52.8	41.5	85.8	53.9	73.2	36.4	53.2
Gemma-4-E4B	53.7	60.6	33.0	34.4	67.8	47.8	46.9	31.9	47.0
Llama-3.1-8B	47.8	39.3	42.6	29.0	67.3	43.8	60.2	39.7	46.2

(a) Turn-level model performance across reasoning topologies and difficulty levels. Each cell reports accuracy for a model under one topology–difficulty condition.



(b) Entailment accuracy as a function of reasoning depth. The x-axis corresponds to the number of inferential steps in the generated structure, and lines show how each model’s performance changes with depth.

Figure 1: Model performance on the canonical DERELAB evaluation set. Panel (a) summarizes aggregate accuracy across topology–difficulty conditions, while Panel (b) shows how performance varies with reasoning depth.

8B across all conditions, consistent with the general scaling trend observed in reasoning benchmarks. Finally, as shown in Figure 1b, the accuracy has an overall decreasing trend with increasing node length or complexity across all models with some outliers.

5.2 Confirmation Bias in Belief Updating

Do LLMs exhibit confirmation bias when updating beliefs under incongruent evidence, and what does this reveal about the nature of their belief revision?

We apply the congruence-condition framework described in §3.8 to all Phase 1 inference results, yielding between 9,029 and 11,682 labeled turns per model across all reasoning types and difficulty tiers. Statistical methodology follows Appendix H; full per-condition counts and supplementary figures appear in Appendix H.

Confirmation bias is near-universal. Every model exhibits statistically significant confirmation bias on default reasoning ($p < 0.001$, all raw McNemar tests), and all but one entry are significant on inheritance (Table 3, Figure 2). The magnitude spans a wide range: from Gemma-4-31B (BiasGap=0.088, OR=18.1) to GPT-5.1 (Bias-

Model	Default		Inheritance		Anch.
	Gap	OR	Gap	OR	
GPT-5.1	0.935	891	0.246	9.3	0.433
GPT-5-mini	0.668	27	0.003	—	0.020
Qwen3-32B [†]	0.578	17	0.065	3.7 [‡]	0.020
Llama-3.1-70B	0.548	13	0.364	7.8	0.189
Gemma-4-E4B	0.597	73	0.324	4.0	0.396
Llama-3.1-8B	0.224	2.8	0.484	8.3	0.119
Gemma-4-31B	0.088	18	0.249	6.6	0.047

Table 3: Confirmation bias results across default and inheritance reasoning. **Gap** = BiasGap (C1 acc – C2 acc); **OR** = odds ratio (bootstrap 95 % CI in Appendix H); **Anch.** = anchoring rate on ground-truth-flipping default sections. [†] Qwen3-32B run in thinking mode. [‡] Not significant after Holm correction ($p = 0.149$); all other non-dashed entries significant at $p < 0.001$ (McNemar, Holm–Bonferroni corrected). OR undefined (—) when C1 errors = 0.

Gap=0.935, OR=891, 95 % CI[590,1487]) on default reasoning. GPT-5.1’s C2 accuracy of 3.5 % means it almost *never* correctly updates its belief from *yes* to *no* when presented with a weakening update on a default chain — a near-total failure of incongruent belief revision.

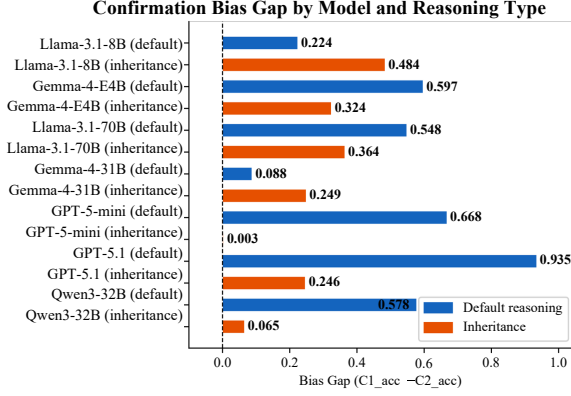


Figure 2: Confirmation bias gap (C1 acc – C2 acc) per model and reasoning type. All default reasoning entries are significant ($p < 0.001$, McNemar). * Qwen3-32B inheritance is the only non-significant entry ($p = 0.149$). OR undefined for GPT-5-mini inheritance (C1 errors = 0).

Default reasoning is systematically harder.

Confirmation bias is consistently stronger on default reasoning than inheritance reasoning for every model (Table 3). The contrast is most dramatic for GPT-5-mini ($0.668 \rightarrow 0.003$) and Qwen3-32B ($0.578 \rightarrow 0.065$, the only non-significant inheritance entry). We attribute this asymmetry to the richer update vocabulary of default reasoning — source-priority conflicts, trap distractors, and multi-turn exception chains create more opportunities for prior-belief anchoring than the structurally simpler inheritance updates.

Thinking mode partially mitigates bias but does not eliminate it. Qwen3-32B, the only model evaluated with explicit chain-of-thought generation, shows near-zero inheritance bias ($p = 0.149$, non-significant) and the lowest anchoring rate of any model on both default (0.020) and inheritance (0.005), suggesting that thinking through the reasoning chain suppresses global belief anchoring. However, its default BiasGap of 0.578 is comparable to standard instruction-tuned models of similar scale, demonstrating that thinking mode does not protect against the per-turn confirmation bias induced by long defeasible chains. The benefit of explicit reasoning is therefore task-specific: it helps where the inference structure is simpler (inheritance) but offers limited protection where multi-chain updates accumulate over many turns (default).

Metacognitive dissociation: knowing but not outputting. A key pattern emerges from the

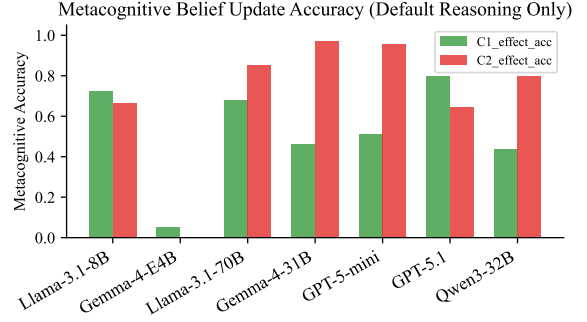


Figure 3: Metacognitive accuracy on the belief-update task (default reasoning only). **C1-effect**: fraction of congruent turns correctly labelled *strengthening*. **C2-effect**: fraction of incongruent turns correctly labelled *weakening*. GPT-5-mini, Qwen3-32B, and Gemma-4-31B show high C2-effect accuracy despite low entailment C2 accuracy, demonstrating the know-but-don’t-output dissociation.

belief-update task (Figure 12). Several models correctly identify that a C2 update is a *weakening* at high rates, yet fail to apply this judgment to their entailment answer. The dissociation is most pronounced in GPT-5-mini (C2-effect accuracy = 95.4 %, entailment C2 accuracy = 20.3 %) and Qwen3-32B (C2-effect accuracy = 79.7 %, entailment C2 accuracy = 30.5 %). Gemma-4-31B shows the same pattern at higher accuracy (C2-effect = 97.0 %, BiasGap = 0.088). This *know-but-don’t-output* dissociation indicates that confirmation bias in these models operates as a belief-revision suppression effect rather than a failure to encode the evidential direction of the update.

Anchoring and semantic override. Across ground-truth-flipping sections, GPT-5.1 maintains its initial answer on 43.3 % of default conversations where the correct answer changed — the highest anchoring rate of any model. Gemma-4-E4B shows the most persistent anchoring on inheritance (tree Hard: 87.8 %), while Gemma-4-31B and GPT-5-mini exhibit near-zero anchoring (≤ 5 %), indicating they do eventually revise their beliefs over a full conversation even though they are biased on individual turns. Separately, Llama-3.1-70B shows the highest semantic override rate: 83.9 % of default C3 turns are incorrectly labelled as *weakening* even when the chain is already defeated, indicating strong sensitivity to surface-level negative language independent of logical scope. Full semantic override and anchoring figures appear in Appendix H.

5.3 Confidence and Inference Path Length

Is model confidence in the correct answer correlated with the length of the inference path required to process an update?

We analyze tree-inheritance turns along two dimensions: *on-path* turns, where the update is causally relevant to the hypothesis, and *off-path* turns, where the update concerns a node outside the hypothesis subject’s inheritance path and should have no effect on confidence. We measure $\Delta_{\text{gold_conf}}$ — the change in probability assigned to the correct label relative to the model’s initial answer — as our primary confidence signal. Full statistical details appear in Appendix G.

Model	Weaken acc. (%)		Off-path	
	Easy	Hard	Intr.	Disc.
GPT-5.1	93.2	72.7	0.9 %	1.50×
Gemma-4-31B	94.5	85.1	0.3 %	1.50×
Llama-3.1-70B	89.0	46.1	15.2 %	0.40×
Llama-3.1-8B	58.9	42.9	36.2 %	0.53×
Gemma-4-E4B	37.0	13.6	4.2 %	1.02×

Table 4: Weaken-turn accuracy and off-path behaviour (Easy). Intrusion (**Intr.**): proportion of off-path turns where the model incorrectly changes its answer. Discrimination Signal (**Disc.**): on-path $|\Delta_{\text{conf}}|$ / off-path $|\Delta_{\text{conf}}|$; below 1× means the model responds more strongly to irrelevant updates.

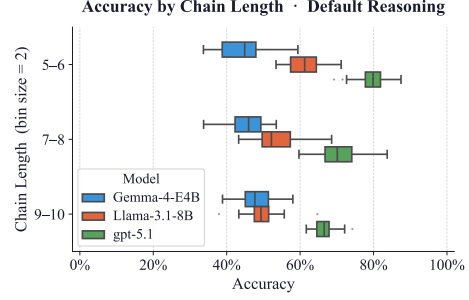
On-path: confidence degrades with chain length.

Longer inference chains cause statistically significant confidence erosion in instruction-tuned models: Spearman correlations on $\Delta_{\text{gold_conf}}$ are significantly negative for Gemma-4-31B ($\rho = -0.298, p < 0.001$), Llama-3.1-70B ($\rho = -0.237, p < 0.001$), and Llama-3.1-8B ($\rho = -0.171, p = 0.003$) on Hard, confirmed by mixed-effects regression ($\beta \in [-0.047, -0.060]$, all $p < 0.001$). GPT-5.1 shows no significant degradation. The effect is most severe on *weaken turns* (Table 4): Llama-3.1-70B drops from 89.0 % to 46.1 % Easy→Hard; Gemma-4-E4B from 37.0 % to 13.6 %.

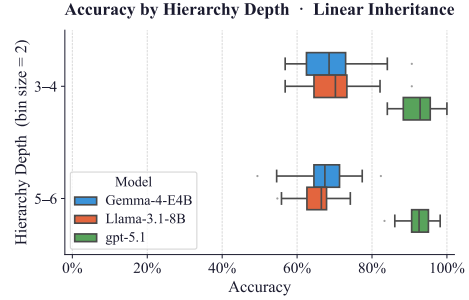
5.4 Seeded Sample Variance Analysis

Are DERELAB evaluations reproducible across independent random seeds, and does stability hold across model tiers?

We run 20 independent seeds per configuration across Default and Linear Inheritance Easy tiers for three representative models, reporting accuracy distributions per seed. Figure 4 shows the result-



(a) Variance in accuracy by property chain length category for default reasoning.



(b) Variance in accuracy by inheritance hierarchy depth for inheritance reasoning (linear).

Figure 4: Reseeding robustness analysis.

ing boxplots; full statistical details appear in Appendix F.

Boxes are consistently tight across all six model–configuration cells (ICC = 0.944, 95 % CI [0.935, 0.959]), and Kruskal–Wallis tests find no significant difference across seeds in any cell (min $p = 0.327$), confirming that DERELAB evaluations are statistically reproducible under reseeding.

6 Conclusion

In this paper, we presented DeReLab, a structural framework designed to evaluate the defeasible reasoning capabilities of Large Language Models. By decomposing reasoning processes into distinct structures, our framework provides a flexible platform for benchmarking how models handle contradictory or updating information. We were successful in showing the lack of robustness for LLMs in cognitive bias like confirmation bias. Looking ahead, future work will focus on incorporating a wider array of non-monotonic reasoning variations into the defeasibility test suite. Along with it, other cognitive science-inspired experiments can be integrated into the framework that will give even more insight into the limitations of models in defeasible tasks.

Limitations

The current framework evaluates the default and inheritance topologies, which can be further expanded to support a broader spectrum of defeasible reasoning structures. To mitigate data contamination, DeReLab relies entirely on pseudowords. So, further study can reflect on how models perform given real-world entities.

Moreover, DeReLab didn't evaluate models' consistency against human performance, leaving it an open question whether human reasoning diverges from algorithmic conclusions. In addition, it was evaluated with two reasoning models. More models could help us find patterns. We found biases exist but didn't discuss why they were found.

References

- Ali Al-Lawati, Jason Lucas, Dongwon Lee, and Suhang Wang. 2026. [Llm benchmark datasets should be contamination-resistant](#). *Preprint*, arXiv:2605.19999.
- Emily Allaway, Jena D. Hwang, Chandra Bhagavatula, Kathleen McKeown, Doug Downey, and Yejin Choi. 2023. [Penguins don't fly: Reasoning about generics through instantiations and exceptions](#). In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pages 2618–2635, Dubrovnik, Croatia. Association for Computational Linguistics.
- Emily Allaway and Kathleen McKeown. 2025. [Evaluating defeasible reasoning in LLMs with DEFREASING](#). In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 10540–10558, Albuquerque, New Mexico. Association for Computational Linguistics.
- Nicholas Asher and Michael Morreau. 1991. Common-sense entailment: A modal theory of nonmonotonic reasoning. In *Logics in AI*, pages 1–30, Berlin, Heidelberg. Springer Berlin Heidelberg.
- Nick Chater, Mike Oaksford, Ulrike Hahn, and Evan Heit. 2011. [Inductive logic and empirical psychology](#). In Dov M. Gabbay, Stephan Hartmann, and John Woods, editors, *Inductive Logic*, volume 10 of *Handbook of the History of Logic*, pages 553–624. North-Holland.
- Stanley F Chen and Joshua Goodman. 1999. An empirical study of smoothing techniques for language modeling. *Computer Speech & Language*, 13(4):359–394.
- Peter Clark, Oyvind Tafjord, and Kyle Richardson. 2020. Transformers as soft reasoners over language. *arXiv preprint arXiv:2002.05867*.
- Jessica Maria Echterhoff, Yao Liu, Abeer Alessa, Julian McAuley, and Zexue He. 2024. [Cognitive bias in decision-making with LLMs](#). In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 12640–12653, Miami, Florida, USA. Association for Computational Linguistics.
- Isabel O. Gallegos, Ryan A. Rossi, Joe Barrow, Md. Mehrab Tanjim, Sungchul Kim, Franck Dernoncourt, Tong Yu, Ruiyi Zhang, and Nesreen Ahmed. 2023. [Bias and fairness in large language models: A survey](#). *Computational Linguistics*, 50:1097–1179.
- Reto Gubelmann, Ioannis Katis, Christina Niklaus, and Siegfried Handschuh. 2023. [Capturing the varieties of natural language inference: A systematic survey of existing datasets and two novel benchmarks](#). *J. of Logic, Lang. and Inf.*, 33(1):21–48.
- Thilo Hagendorff, Sarah Fabi, and Michal Kosinski. 2022. [Human-like intuitive behavior and reasoning biases emerged in large language models but disappeared in chatgpt](#). *Nature Computational Science*, 3:833 – 838.
- John F. Horty, Richmond H. Thomason, and David S. Touretzky. 1990. [A skeptical theory of inheritance in nonmonotonic semantic networks](#). *Artificial Intelligence*, 42(2):311–348.
- Erik Jones and Jacob Steinhardt. 2022. Capturing failures of large language models via human cognitive biases. In *Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS '22*, Red Hook, NY, USA. Curran Associates Inc.
- Douwe Kiela, Max Bartolo, Yixin Nie, Divyansh Kaushik, Atticus Geiger, Zhengxuan Wu, Bertie Vidgen, Grusha Prasad, Amanpreet Singh, Pratik Ringshia, and 1 others. 2021. Dynabench: Rethinking benchmarking in nlp. In *Proceedings of the 2021 conference of the North American chapter of the Association for Computational Linguistics: human language technologies*, pages 4110–4124.
- Jemma König, Andreea S Calude, and Averil Coxhead. 2020. Using character-grams to automatically generate pseudowords and how to evaluate them. *Applied Linguistics*, 41(6):878–900.
- Terry K. Koo and Mae Y. Li. 2016. [A guideline of selecting and reporting intraclass correlation coefficients for reliability research](#). *Journal of Chiropractic Medicine*, 15(2):155–163.
- Vladimir Lifschitz. 1988. Benchmark problems for formal nonmonotonic reasoning: Version 2.00. In *International Workshop on Non-Monotonic Reasoning*, pages 202–219. Springer.
- Vladimir Lifschitz. 1989. Benchmark problems for formal nonmonotonic reasoning. In *Proceedings of the 2nd International Workshop on Non-Monotonic Reasoning*, page 202–219, Berlin, Heidelberg. Springer-Verlag.

- Andrew Liu, Henry Prior, Gargi Balasubramaniam, Rivka Moroshko, Amir Zait, Ilia Labzovsky, Danny Karmon, Ishita Dasgupta, Kimberly Stachenfeld, and Kenneth Marino. 2025. [Recoglab: a framework testing relational reasoning & cognitive hypotheses on llms](#). In *International Conference on Representation Learning*, volume 2025, pages 45077–45106.
- Olivia Macmillan-Scott and Mirco Musolesi. 2024. [\(ir\)rationality and cognitive biases in large language models](#). *Preprint*, arXiv:2402.09193.
- Inbal Magar and Roy Schwartz. 2022. [Data contamination: From memorization to exploitation](#). In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 157–165, Dublin, Ireland. Association for Computational Linguistics.
- Boris New, Jessica Bourgin, Julien Barra, and Christophe Pallier. 2024. Unipseudo: A universal pseudoword generator. *Quarterly Journal of Experimental Psychology*, 77(2):278–286.
- Raymond Nickerson. 1998. [Confirmation bias: A ubiquitous phenomenon in many guises](#). *Review of General Psychology*, 2:175–220.
- Andreas Opedal, Alessandro Stolfo, Haruki Shirakami, Ying Jiao, Ryan Cotterell, Bernhard Schölkopf, Abulhair Saparov, and Mrinmaya Sachan. 2024. Do language models exhibit the same cognitive biases in problem solving as human learners? In *Proceedings of the 41st International Conference on Machine Learning*, ICML’24. JMLR.org.
- Mihir Parmar, Nisarg Patel, Neeraj Varshney, Mutsumi Nakamura, Man Luo, Santosh Mashetty, Arindam Mitra, and Chitta Baral. 2024. [LogicBench: Towards systematic evaluation of logical reasoning ability of large language models](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13679–13707, Bangkok, Thailand. Association for Computational Linguistics.
- Stephen Pilli. 2024. [Exploring conversational agents as an effective tool for measuring cognitive biases in decision-making](#). *Preprint*, arXiv:2401.06686.
- David Poole. 1988. [A logical framework for default reasoning](#). *Artificial Intelligence*, 36(1):27–47.
- R. Reiter. 1980. [A logic for default reasoning](#). *Artificial Intelligence*, 13(1):81–132. Special Issue on Non-Monotonic Logic.
- Raymond Reiter. 1978. *On Closed World Data Bases*, pages 55–76. Springer US, Boston, MA.
- Rachel Rudinger, Vered Shwartz, Jena D. Hwang, Chandra Bhagavatula, Maxwell Forbes, Ronan Le Bras, Noah A. Smith, and Yejin Choi. 2020. [Thinking like a skeptic: Defeasible inference in natural language](#). In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4661–4675, Online. Association for Computational Linguistics.
- Samuel Schmidgall, Carl Harris, Ime Essien, Daniel Olshvang, Tawsifur Rahman, Ji Woong Kim, Rojin Ziaei, Jason Eshraghian, Peter Abadir, and Rama Chellappa. 2024. [Addressing cognitive bias in medical language models](#). *Preprint*, arXiv:2402.08113.
- Karan Singhal, Shekoofeh Azizi, Tao Tu, S. Sara Mahdavi, Jason Wei, Hyung Won Chung, Nathan Scales, Ajay Tanwani, Heather Cole-Lewis, Stephen Pfohl, Perry Payne, Martin Seneviratne, Paul Gamble, Chris Kelly, Nathaneal Scharli, Aakanksha Chowdhery, Philip Mansfield, Blaise Agüera y Arcas, Dale Webster, and 11 others. 2022. [Large language models encode clinical knowledge](#). *Preprint*, arXiv:2212.13138.
- Oyvind Tafjord, Bhavana Dalvi, and Peter Clark. 2021. Proofwriter: Generating implications, proofs, and abductive statements over natural language. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 3621–3634.
- Alaina N. Talboy and Elizabeth Fuller. 2023. [Challenging the appearance of machine intelligence: Cognitive bias in llms and best practices for adoption](#). *Preprint*, arXiv:2304.01358.
- Tristan Thrush, Kushal Tirumala, Anmol Gupta, Max Bartolo, Pedro Rodriguez, Tariq Kane, William Gavidia Rojas, Peter Mattson, Adina Williams, and Douwe Kiela. 2022. Dynatask: A framework for creating dynamic ai benchmark tasks. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, pages 174–181.
- Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Gehrmann, Prabhajan Kambadur, David Rosenberg, and Gideon Mann. 2023. [Bloomberggpt: A large language model for finance](#). *Preprint*, arXiv:2303.17564.
- Zhentao Xie, Jiabao Zhao, Yilei Wang, Jinxin Shi, Yanhong Bai, Xingjiao Wu, and Liang He. 2024. [Mindscope: Exploring cognitive biases in large language models through multi-agent systems](#). In *European Conference on Artificial Intelligence*.
- Yeliang Xiu, Zhanhao Xiao, and Yongmei Liu. 2022. [LogicNMR: Probing the non-monotonic reasoning ability of pre-trained language models](#). In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 3616–3626, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.
- Hitomi Yanaka, Koji Mineshima, Daisuke Bekki, Kentaro Inui, Satoshi Sekine, Lasha Abzianidze, and Johan Bos. 2019a. [Can neural networks understand monotonicity reasoning?](#) In *Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 31–40, Florence, Italy. Association for Computational Linguistics.

Hitomi Yanaka, Koji Mineshima, Daisuke Bekki, Kentaro Inui, Satoshi Sekine, Lasha Abzianidze, and Johan Bos. 2019b. [HELP: A dataset for identifying shortcomings of neural models in monotonicity reasoning](#). In *Proceedings of the Eighth Joint Conference on Lexical and Computational Semantics (*SEM 2019)*, pages 250–255, Minneapolis, Minnesota. Association for Computational Linguistics.

Jiayi Ye, Yanbo Wang, Yue Huang, Dongping Chen, Qihui Zhang, Nuno Moniz, Tian Gao, Werner Geyer, Chao Huang, Pin-Yu Chen, Nitesh V Chawla, and Xiangliang Zhang. 2025. [Justice or prejudice? quantifying biases in LLM-as-a-judge](#). In *The Thirteenth International Conference on Learning Representations*.

Wenting Zhao, Justin Chiu, Claire Cardie, and Alexander Rush. 2023. [Abductive commonsense reasoning exploiting mutually exclusive explanations](#). In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14883–14896, Toronto, Canada. Association for Computational Linguistics.

Hanyang Zhong, Liman Wang, Wenting Cao, and Zeyuan Sun. 2024. [Balancing rigor and utility: Mitigating cognitive biases in large language models for multiple-choice questions](#).

Kun Zhou, Yutao Zhu, Zhipeng Chen, Wentong Chen, Wayne Xin Zhao, Xu Chen, Yankai Lin, Ji-Rong Wen, and Jiawei Han. 2023. [Don’t make your llm an evaluation benchmark cheater](#). *Preprint*, arXiv:2311.01964.

A DERELAB Framework

As shown in Figure 5, different modules of Derelab Framework are explained in this section.

A.1 Graph Generation

Our reasoning structure had 2 types of nodes: entity and property nodes. The nodes had 4 different types of edges connecting them. Following:

- **Hypothesis Edge:** Represents the target query or tentative conclusion that is currently under evaluation.
- **Has Attribute:** Denotes factual and monotonic assignment of a property to an entity.
- **Defeasible Has Property:** Represents a non-monotonic implication indicating that an entity “generally” or “typically” possesses a certain property. For example, “Red objects are generally cubic”.
- **Defeasible Lack Property:** Represents a non-monotonic implication indicating that an entity “generally” does not possess a specific trait. For example, “Cubic objects generally do not roll”.

The graph generator module could construct three distinct topologies:

Linear Inheritance: The graph formed a single, unbranched entity chain from a root concept down to a terminal subclass. The root concept was assigned a base target property. To introduce non-monotonicity, defeasible edges were probabilistically injected at intermediate nodes in the chain. The final hypothesis strictly queried the leaf node.

Tree Inheritance: The tree expanded from a root node where the maximum branching factor exponentially decayed with depth. Branches also underwent random survival checks, resulting in asymmetrical structures. Defeasible edges were randomly added across intermediate nodes, and the hypothesis edge targeted a leaf node.

In Table 6, the parameters tuned to set the difficulty of Inheritance Reasoning topologies across different presets are listed: easy (fewer nodes, less depth and breadth) and hard.

Default Reasoning: This topology modelled a main property chain. Multiple entities could be connected to a root property. This root property implied subsequent properties. To evaluate robustness against noise, irrelevant properties (distractor

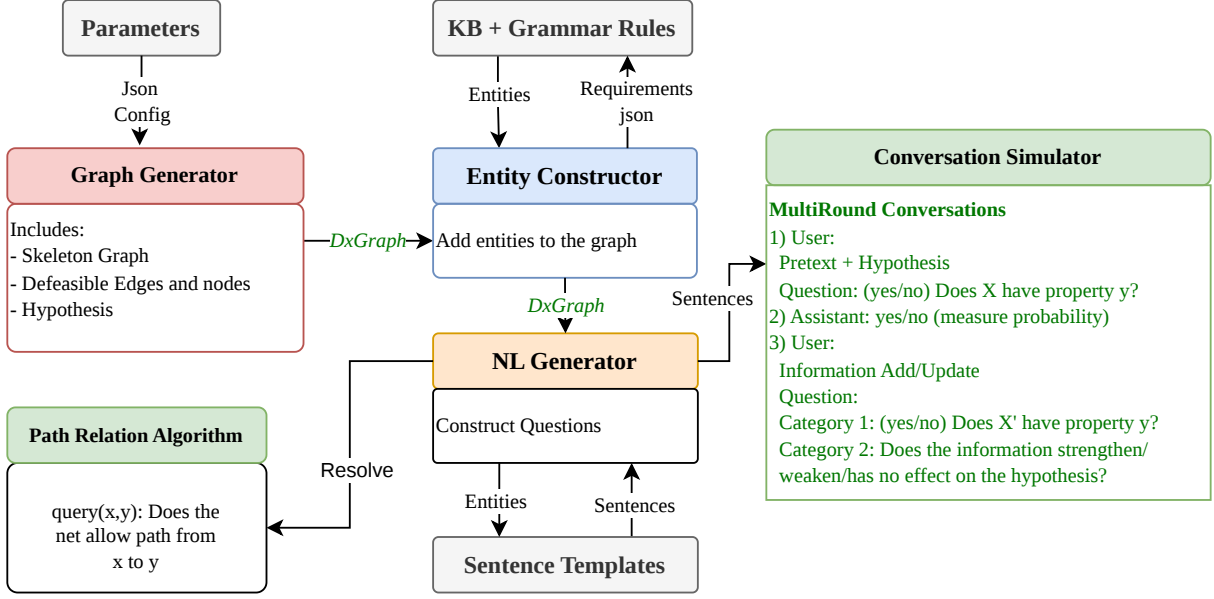


Figure 5: Different Modules in Derelab Framework

nodes) were attached directly to random entities. Defeasible edges were injected onto intermediate property nodes and assigned a depth-based serial number. This serialization ensured logical order from general to specific used later translating into conversation. The hypothesis then queried a terminal property leaf.

In Table 6, the parameters tuned to set the difficulty of default reasoning topology are listed: easy (fewer properties, objects, and hypotheses) and hard.

A.2 Entity Construction

The abstract graph nodes and edges from the graph generator module were populated with nonce entities and coherent attributes. Entity and attribute preparations have been discussed in Appendix D.

The property sampling was *domain-aware*. A root concept dictated the pool of valid attributes (e.g., a “Tool” domain permits attributes like *material* and *utility*, whereas an “Animal” domain would permit *habitat*). Additionally, distant attributes were explicitly sampled to act as irrelevant attributes in default reasoning.

Template Structures: To generate grammatically sound sentences, properties were represented using a structured template. Each attribute category contained a pool of candidate values along with two linguistic templates (singular and plural) to support different relational contexts.

The abstract template structure was defined as follows:

Attribute Category: <String>

Examples: [<Value_1>, <Value_2>, ..., <Value_n>]

Template: "<Plural Verb Phrase> [value]"

Object Template: "<Singular Verb Phrase> [value]"

An Example: To illustrate the property instantiation, consider a graph requiring an entity assignment and a property implication within the “Tool” domain.

At first, the “material” attribute was sampled for the “Tool” domain.

Example Value Selected: Titanium

Template: are made of [value]

Object Template: is made of [value]

Then, the placeholder was replaced, generating the exact predicate structure passed to the graph populator:

Attribute Name: material

Selected Value: Titanium

Filled Template: are made of Titanium

Filled Object Predicate: is made of Titanium

A.3 Path Resolver Algorithm

To determine the ground truth entailment of the hypotheses, the framework employed a path-based non-monotonic reasoning algorithm, based on the Parallel Marker Propagation Machine (PMPM) framework introduced by (Horty et al., 1990). This algorithm relies on a bottom-up (forward chaining)

Table 5: Configuration Parameters for Easy and Hard Difficulties in Inheritance Reasoning Topology

Parameter	Easy	Hard
Global Settings		
Sparsity factor range	0.50–0.80	0.40–0.60
Serial random	True	True
Jellyfish Preset		
Max depth	4	8
Root breadth	5	13
Branching (min–max)	1–1	1–1
Survival probability	0.40	0.60
Branching decay	1.0	1.0
Expected nodes	4–8	24–50
Balanced Preset		
Max depth	3	4
Root breadth	2	3
Branching (min–max)	2–2	2–2
Survival probability	1.0	1.0
Branching decay	1.0	1.0
Expected nodes	7	46
Bushy Preset		
Max depth	2	4
Root breadth	3	4
Branching (min–max)	1–2	1–4
Survival probability	0.70	0.80
Branching decay	1.0	0.75
Expected nodes	5–9	20–60

Table 6: Comparison of Easy and Hard Parameters in Default Reasoning Topology

Parameter	Easy	Hard
Chain length	5–10 nodes	10–20 nodes
Num objects	2–3	4–6
Irrelevant attributes	1–2	2–4
Sparsity factor	0.30–0.50	0.40–0.70
Max hypothesis sections	2	3

approach to argument construction. It resolves contradictory inheritance paths by prioritising specific evidence over generalisations.

Before resolution, the logical topology was abstracted into a signed Directed Acyclic Graph (DAG), where edges denoted either positive support (supporting links) or negative evidence (defeating links). Some edges from the topology were ignored.

- **Supporting Links(+):** “inheritance”, “is_a”, “implies”, “has_property”, and “defeasible_has_property” edges.
- **Defeating Links(-):** “defeasible_lacks_property” edges.
- **Ignored Links(o):** “hypothesis” and “has_attribute” edges.

The evaluation then proceeded through the following core phases:

- **Trimming:** The graph was first pruned to isolate the relevant nodes and edges to the specific query. The algorithm first performed a forward search from the source entity along strictly positive paths, followed by a backward search from the target property. Only nodes residing in the intersection of these valid paths were retained for evaluation.
- **Propagation:** The trimmed network was then evaluated in a strict topological order. This ensured that every intermediate node was fully resolved before its logical state was propagated forward. The algorithm maintained states of proven and defeated properties, carrying the deductive logic upward from the source entity.
- **Direct Preemption:** Any evidence directly attached to the root subject entity was treated as absolute, immediately overriding any longer, inherited reasoning chains.
- **Specificity Based Conflict Resolution:** When an intermediate node received conflicting logics (e.g., inheriting both a property and its negation from different parent nodes), the algorithm inspected the hierarchical relationship between the conflicting sources. If a positive path existed from source A to source B, A was identified as a more specific subclass of B. The logic originating from a more specific subclass strictly preempted the signal from a superclass.
- **Skeptical Inference:** If conflicting deductive paths were completely symmetric, meaning neither source node was a subclass of the other, the algorithm classified the property’s state as unknown.
- **Default Chain Resolution:** For graphs representing linear property chains, the algorithm applied a “defaults carry through” rule. Entailment flowed through the chain unless explicitly blocked by negative defeasible evidence. Furthermore, the framework supported synthetic bypass rules, allowing the logic to skip intermediate nodes if sufficient, unblocked entry points were established earlier in the chain.

The execution of the formal PMPM strategy from (Horty et al., 1990) are detailed throughout Algorithms 1, 2, and 3.

The final computed state of the target property node: entailed (True), defeated (False), or undetermined (Unknown).

A.4 Natural Language Generation

To translate the populated graph into readable text, a natural language generation module mapped each directed edge to a coherent English sentence. This was achieved using pre-defined structural templates that defined the grammatical interaction between a source node and a target node based on the edge type.

Common sentence templates include:

- Inheritance (*is_a*): *{subject} is a {object}*.
- Property Assignment (*has_attribute*): *{subject} {object_predicate}*.
- Implication (*implies*): *Objects that {subject_predicate} generally {object_predicate_plural}*.

During sentence generation, the module injected the instantiated entity labels and filled property predicates into these sentence templates. For example, given a nonce entity (*Zibber*), a base concept (*Tool*), and an instantiated property (*made of Titanium*), the graph edges are translated as follows:

- Inheritance Edge Translation:
Mapping: {subject: Zibber} is a {object: Tool}.
Final Output: “The Zibber is an Instrument.”
- Property Assignment Edge Translation:
Mapping: {subject: Zibber} {object_predicate: is made of Titanium}.
Final Output: “The Zibber is made of Titanium.”
- Implication Edge Translation:
Mapping: Objects that {subject_predicate: are Tools} generally {object_predicate_plural: are made of Titanium}.
Final Output: “Objects that are Tools generally are made of Titanium.”

A.5 Conversation Simulation

After the natural language realisation of every graph edge, a module determined the order in

which the graph’s reasoning structure would be progressively disclosed to the resolver, an algorithm to evaluate reasoning against the hypothesis.

An initial premise was first constructed from the deterministic backbone of the graph: consisting of the “inheritance” and “has-attribute” edges. This premise, along with the hypothesis, was presented to the resolver before any defeasible evidence was introduced. It established the base structure against which all subsequent updates would be evaluated.

Defeasible edges were then revealed according to a controlled schedule. In the default configuration, edges were disclosed in ascending order of their assigned serial indices. An alternative randomised reveal mode was also supported to mitigate the risk of models exploiting positional patterns rather than inferential reasoning. In that mode, defeasible edges for a given subject were shuffled into a random reveal order. The resolver was supplied only with the edges explicitly revealed up to that point.

After each defeasible edge was introduced, the hypothesis was re-evaluated against the accumulated evidence. This re-evaluation resulted in one of three outcomes: strengthening, weakening, or undetermined. The effect of each reveal was computed: positive defeasible evidence was classified as strengthening the hypothesis, while negative evidence or exceptions were classified as weakening it. Cases in which the state remained unchanged were assigned a no-effect label. Thus, we got two labels for each turn: the updated hypothesis verdict and the effect of the most recent reveal.

B Default Reasoning

In this section, we have discussed which of the benchmark problems from (Lifschitz, 1988) have been addressed by every question type in Default reasoning.

Type 1: Direct Chain Manipulation

Consider an initial premise:

“Heavy blocks are normally located on the table.”

- *In Basic Default Reasoning* (Benchmark A1), introducing the fact:

“Block A is heavy”

leads to the conclusion:

“Block A is on the table.”

Algorithm 1 Graph Trimming (trim-for-query)

Require: Graph Γ , source node x , target node y

Ensure: Relevant trimmed subgraph Γ'

- 1: $ForwardSet \leftarrow$ nodes reachable from x via strictly positive links
 - 2: $BackwardSet \leftarrow$ nodes from which y can be reached
 - 3: $\Gamma' \leftarrow ForwardSet \cap BackwardSet$
 - 4: **return** Γ'
-

Algorithm 2 Degree Ordering (select-next-degree)

Require: Trimmed Graph Γ' , set of processed nodes P

Ensure: Set of nodes for the next topological layer N

- 1: $N \leftarrow \emptyset$
 - 2: **for all** node $v \in \Gamma'$ such that $v \notin P$ **do**
 - 3: **if** all predecessors of v are contained in P **then**
 - 4: $N \leftarrow N \cup \{v\}$
 - 5: **end if**
 - 6: **end for**
 - 7: **return** N
-

- *Default Reasoning with Several Defaults* (Benchmark A3) extends it for several defaults.
- *Default Reasoning with a Disabled Default* (Benchmark A4) tests the revocation of this conclusion by introducing an overriding fact: “Block A is an exception to this rule.”

The NL Generator traced the primary logical chain. Introducing new facts either reinforced the current hypothesis or broke the chain. When generating a negative update, NL Generator randomly alternated between a *direct negation* (“The object does not have the property”) and *exception phrasing* (“The object is an exception to the rule”). If the parent category has already been established as negative earlier in the chat, the system forcefully overrode the exception phrasing to prevent contradiction.

Type 2: Irrelevant Information

To understand *Default Reasoning with Irrelevant Information* (Benchmark A2), consider the generator has concluded:

“Block A is on the table”.

Introducing a new, unrelated fact such as

“Block A is red”,

should not affect the prior conclusion.

The NL Generator generated two distinct variants of irrelevance to probe the model’s robustness. “Safe” irrelevance introduced completely benign

and unrelated traits. “Trap” irrelevance dynamically scanned the active logic graph and synthesised distractor properties that share overlapping vocabulary or negative prefixes with valid chain nodes. In all Type 2 updates, the programmed ground-truth registered “no effect” on the underlying hypothesis.

Type 3: Unknown Exceptions

This type introduces disjunctive and incomplete knowledge.

- In *Reasoning about Unknown Exceptions I* (Benchmark A6), the disjunctive premise points towards object instantiations. For instance,

“At least one of Block A or Block B is not on the table.”

At this point, the exact status of Block A is undetermined. If subsequent evidence reveals “Block A is on the table,”

the system must deduce that the exception is Block B.

- In *Reasoning about Unknown Exceptions III* (Benchmark A8), a generic statement is used. For instance,

“At least one heavy block is not on the table. ”

Afterwards, if we reveal the fact:

“Block A is Heavy”

then the conclusion should be:

Algorithm 3 Main Skeptical Inference Procedure (query)

Require: Graph Γ , subject x , target property y

Ensure: Entailment state of y (True, False, or Unknown)

```
1:  $\Gamma' \leftarrow \text{TrimGraph}(\Gamma, x, y)$ 
2:  $\text{SortedNodes} \leftarrow \text{TopologicalSort}(\Gamma')$   $\triangleright$  Process parents before children
3:  $\text{State}(x) \leftarrow \text{True}$ 
4: for all node  $w \in \text{SortedNodes}$  do  $\triangleright$  1. Direct Evidence (Absolute Truth overrides all)
5:   if direct positive link  $(x \rightarrow w)$  exists in  $\Gamma$  then
6:      $\text{State}(w) \leftarrow \text{True}$ 
7:   else if direct negative link  $(x \not\rightarrow w)$  exists in  $\Gamma$  then
8:      $\text{State}(w) \leftarrow \text{False}$ 
9:   else  $\triangleright$  2. Inherited Evidence (Check parent states)
10:     $\text{PosParents} \leftarrow \{p \mid \text{State}(p) = \text{True} \text{ and positive link } (p \rightarrow w) \text{ exists}\}$ 
11:     $\text{NegParents} \leftarrow \{p \mid \text{State}(p) = \text{True} \text{ and negative link } (p \not\rightarrow w) \text{ exists}\}$ 
12:    if  $\text{PosParents} \neq \emptyset$  and  $\text{NegParents} = \emptyset$  then
13:       $\text{State}(w) \leftarrow \text{True}$ 
14:    else if  $\text{NegParents} \neq \emptyset$  and  $\text{PosParents} = \emptyset$  then
15:       $\text{State}(w) \leftarrow \text{False}$ 
16:    else if  $\text{PosParents} \neq \emptyset$  and  $\text{NegParents} \neq \emptyset$  then  $\triangleright$  3. Conflict: Specific subclass logic
      overrides general superclass logic
17:       $\text{State}(w) \leftarrow \text{ResolveSpecificity}(\text{PosParents}, \text{NegParents})$ 
18:    end if
19:  end if
20: end for
21: return  $\text{State}(y)$ 
```

“Block A is not on the table.”

The NL Generator identified segments of the logic graph where multiple subjects shared a target property, but one or more subjects actually failed to inherit it. Rather than explicitly isolating the failing subject immediately, the generator realized a disjunctive sentence grouping them together. It injected the disjunctive group claim before the individual clues occurred in the timeline. The language model was expected to transition into an “undetermined” state and maintain that until the subsequent individual updates eliminated enough variables to collapse the disjunction.

Type 4: Source-Priority Conflicts

- *Priorities between Defaults* (Benchmark A9) involves conflicting sources (Jack and Mary). If Jack asserts,
“Block A is not on the table,”
while Mary asserts,
“Block A is on the table.”
Because the environment dictates that Mary is a more reliable source than Jack, her claim

takes precedence, and the final conclusion must side with her.

NL Generator evaluated complex belief revision by orchestrating disputes between simulated agents (“Expert A” and “Expert B”). A known priority ordering was announced in the pretext once:

“Expert A is generally more reliable than Expert B.”

The implementation had several mechanisms:

- **Dual Conflict Topologies:** Realization of two distinct shapes of disagreement:

The first is a *direct contradiction*, where the high-priority and low-priority agents argued over the exact same node.

The second is a *structural bypass*. In a bypass, the low-priority agent claimed that a critical connective rule is blocked.

Rather than arguing the point, the high-priority agent introduced a “shortcut” rule that logically routed around the blocked node (ensuring the bypass source index was strictly before the block, and the target index was

strictly after), successfully restoring the hypothesis path.

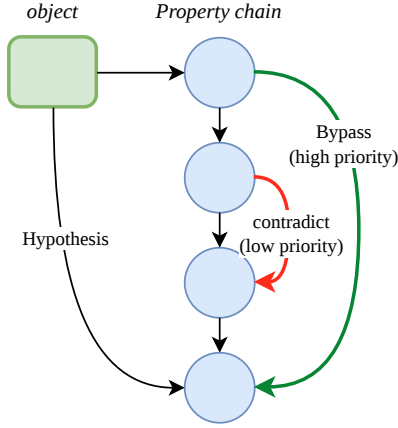


Figure 6: Conflict: Contradict and Bypass

- **Temporal Ordering:** Randomizing the order in which the agents speak:

If the low-priority agent spoke first, the resolver registered a drop in confidence, followed by a nonmonotonic recovery when the high-priority agent corrected the record.

Conversely, if the high-priority agent spoke first, the system acted as an epistemic shield. When the low-priority agent disagreed, the resolver computed “no effect”.

- **Redundancy Filtering:** If a lower-priority source subsequently echoed a confirmation already established by a higher-priority source, the algorithm enforced a strict “no effect”.
- **Subject Isolation:** Entities subjected to Type 4 source-priority conflicts were disjoint from entities undergoing Type 1, 2, or 3 scenarios to ensure the model’s success or failure be a function of priority resolution.

In Table 7, we have briefly discussed the mapping.

C Inheritance Reasoning

Figure 7 shows the real-life hierarchy mimics we have used as presets.

Table 7: Mapping of Question Types to (Lifschitz, 1988)’s Benchmark Equivalents

Question Type	Benchmark Equivalents
Type 1: Direct Chain Manipulation	A1 (Basic Default Reasoning) A3 (Default Reasoning with Several Defaults) A4 (Disabled Default)
Type 2: Irrelevant Information	A2 (Default Reasoning with Irrelevant Information)
Type 3: Scoped Uncertainty and Unknown Exceptions	A6, A8 (Reasoning about Unknown Exceptions)
Type 4: Source-Priority Conflicts	A9, A10 (Priorities between Defaults)

D Data Creation Process

This section contains the overall architecture of the whole data creation process.

D.1 Nonce Word Generation

To populate the entity nodes of the reasoning structure, we generated nonce words using character-level Markov n-gram models trained on different seed corpora following the UniPseudo paradigm of (New et al., 2024). We followed the CGCA approach of (König et al., 2020) to avoid bleed between word-initial and word-final contexts. In this approach, the model’s transitions were bucketed by relative position (initial, medial, final).

Afterwards, transition probabilities were smoothed using interpolated Kneser–Ney following (Chen and Goodman, 1999), which prevents zero-probability assignments on unseen continuations and supports candidate scoring. Each candidate was additionally subject to a legality check (König et al., 2020).

To train the model, we prepared different reference corpora: reference nouns and adjectives, synthetic phonotactic syllables, domains with Datamuse expansion, etc. We also sampled nonce words, locking the suffix for noun and adjective-shaped nonce words. Finally, we added a filter to dump real English words, repeated characters, and unpronounceable consonant strings.

D.2 Property Fetching and Enrichment

At first, we prepared a list of domain names. To fetch properties given the domain names, we queried the [normalised ConceptNet DB](#), [WikiData](#), and [Datamuse API](#). Then we categorised the properties into groups. For instance, if a domain “animal”

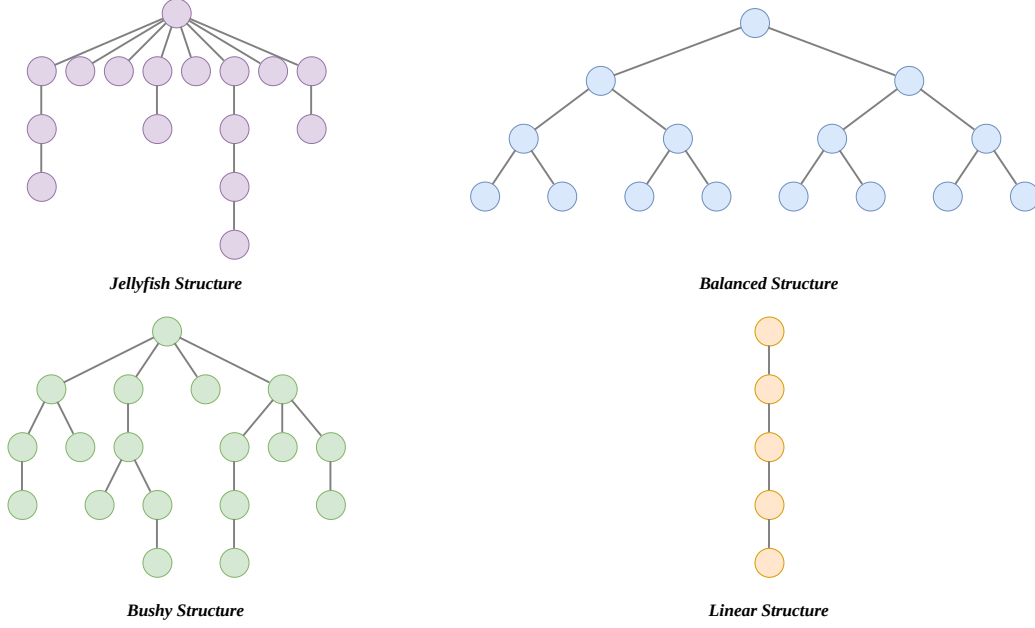


Figure 7: Inheritance: Real Life Hierarchy Mimic

has two properties named “fly” and “swim”, we assigned them to the group of “capability”. Moreover, we used a Large Language Model to augment more properties under each group, augment new property groups, and enrich the domain-attribute mapping.

We represented each such group as “attribute” and properties under a group as “values”. To use these properties in a sentence, we need to prepare templates. For instance, “fly” is a value under the attribute “capability”. Now, the sentence should be “[Subject] can fly”. Here, the object predicate was stored as a template (“can [value]”). Values can be different capabilities in this case.

Domain Name	Attribute Count
Animal	32
Plant	30
Tool	25
Food	24
Vehicle	25
Compound	21
Object	24
Material	22
Microorganism	18
Infrastructure	24

Table 8: Domain Names and their corresponding Attribute Counts

Statistic	Value
ICC(2,1)	0.9442
95 % CI lower bound	0.9348
95 % CI upper bound	0.9594
Number of cells	6
Number of seeds per cell	20
Bootstrap resamples	1,000

Table 9: Pooled ICC(2,1) across all model-configuration cells. An ICC ≥ 0.9 indicates excellent consistency; the narrow confidence interval confirms stable estimation with 20 seeds.

E Data Examples

E.1 Default Reasoning

In Figure 13, an example graph on default reasoning is shown.

E.2 Linear Inheritance

In Figure 14, an example graph on linear inheritance is shown.

E.3 Tree Inheritance

In Figure 15, an example graph on tree inheritance is shown.

F Seeded Sample Variance: Statistical Details

This section provides full statistical details for the seeded sample variance analysis. We report the Intr-

Model	Config	Mean	Std	CV (%)	H	p	Sig.
<i>Default Reasoning, Easy</i>							
gpt-5.1	Default Easy	0.702	0.026	3.73	16.59	0.617	No
Llama-3.1-8B	Default Easy	0.528	0.036	6.90	21.19	0.327	No
Gemma-4-E4B	Default Easy	0.467	0.028	5.96	12.46	0.865	No
<i>Linear Inheritance, Easy</i>							
gpt-5.1	Linear Easy	0.921	0.033	3.62	18.74	0.474	No
Llama-3.1-8B	Linear Easy	0.665	0.046	6.88	21.16	0.328	No
Gemma-4-E4B	Linear Easy	0.683	0.054	7.86	17.26	0.572	No

Table 10: Per-cell Kruskal–Wallis results across 20 seeds. Mean and Std are computed over per-seed accuracy scores. CV = coefficient of variation ($\text{Std} / \text{Mean} \times 100$). H = Kruskal–Wallis statistic; p = asymptotic p -value under χ^2 approximation with 19 degrees of freedom. No cell is significant at $\alpha = 0.05$.

aclass Correlation Coefficient (ICC) pooled across all configurations and per-configuration Kruskal–Wallis tests of seed-level accuracy distributions.

Experimental Setup

We run 20 independent random seeds per configuration, generating 30 conversations per seed. Three models are evaluated: gpt-5.1, Gemma-4-E4B, and Llama-3.1-8B-Instruct, representing frontier closed, open mid-scale, and open small model tiers respectively. Two configurations are tested: *Default Reasoning Easy* and *Linear Inheritance Easy*. Accuracy is computed at the turn level against formally verified ground truth labels for each seed independently.

Intraclass Correlation Coefficient

To quantify the consistency of accuracy scores across seeds, we compute the two-way random effects ICC(2,1) treating seeds as the replicate factor and configurations as the grouping factor. The ICC is estimated on a 6×20 matrix (6 cells = 3 models $\times$ 2 configurations; 20 seeds), with bootstrap confidence intervals computed over 1,000 resamples. The pooled ICC of 0.944 (95 % CI [0.935, 0.959]) indicates *excellent* consistency across seeds under all tested configurations (Koo and Li, 2016). The upper bound of the CI does not fall below 0.93, confirming that this conclusion is robust to bootstrap sampling variability.

Kruskal–Wallis Tests

For each model–configuration cell, we additionally run a Kruskal–Wallis H-test with the null hypothesis that the 20 per-seed accuracy scores are drawn from the same distribution. Unlike the ICC, which measures overall consistency, the Kruskal–Wallis test is sensitive to any systematic ordering or drift across seeds within a cell. A non-significant result

($p > 0.05$) provides direct evidence that no seed produces systematically higher or lower accuracy than others.

All six cells are non-significant (minimum $p = 0.327$, maximum $H = 21.19$), providing no evidence that any seed produces systematically different results within a given configuration. The coefficient of variation remains below 8 % in all cells, with frontier model gpt-5.1 showing the lowest variance ($\text{CV} \leq 3.73\%$) and Gemma-4-E4B on linear inheritance showing the highest ($\text{CV} = 7.86\%$), still well within the range expected from sampling variability alone at $n = 30$ conversations per seed.

Taken together, the ICC and Kruskal–Wallis results confirm that DERELAB evaluations are statistically reproducible under reseeding: different seeds generate statistically equivalent datasets from the same configuration, and no seed produces a systematically advantaged or disadvantaged evaluation.

G Path Confidence Analysis: Full Statistical Details

This appendix provides complete statistical results for the inference-path confidence analysis summarised in §5.3 of the main paper. We report Spearman correlations, mixed-effects regression coefficients, Brier scores, and off-path signal discrimination results separately for on-path and off-path turns. Results for Qwen3-32B and DeepSeek-R1-0528 are excluded from the main analysis owing to incomplete probability outputs under the constrained-decoding protocol.

Methodology

$\Delta\text{gold_conf}$. Raw confidence values conflate graph-level baseline uncertainty with inference-path effects. We therefore compute $\Delta\text{gold_conf} = \text{gold_conf}(\text{update}) - \text{gold_conf}(\text{initial})$, where

$\text{gold_conf}(\text{initial})$ is the probability assigned to the correct label before any updates. Negative Δ indicates confidence erosion.

On-path filtering. Only turns where the update effect is *strengthen* or *weaken* are used for the on-path analysis; these are causally relevant to the hypothesis. Off-path turns (effect = no effect) form the control condition.

Statistics. Spearman ρ measures the monotonic association between inference path length and confidence. Mixed-effects regression ($\Delta\text{gold_conf} \sim \text{distance} + \text{difficulty} + (1 \mid \text{conv_id})$) controls for conversation-level random variation. Cohen’s d compares short ($\text{distance} \leq 2$) versus long ($\text{distance} > 2$) chains.

G.1 On-Path Confidence Degradation

We first analyze updates that lie directly on the active inheritance path between the hypothesis entity and the root property. These updates are causally relevant: a well-calibrated model should respond strongly to them, but confidence should degrade gradually as the number of inferential hops increases.

Table 11a reports Spearman correlations between inference path distance and constrained answer confidence. Negative correlations indicate confidence erosion as reasoning chains become longer. Several open-weight models, including Gemma-4-31B and both Llama variants, show significant negative correlations, especially on the hard split, suggesting that confidence decays systematically with increasing inference depth.

The mixed-effects regressions in Table 11b confirm this trend while controlling for conversation-level variance. Gemma-4-31B and the Llama models exhibit consistently negative coefficients, indicating that each additional hop along the inheritance path reduces confidence in the correct answer. In contrast, GPT-5.1 shows no significant distance effect, suggesting greater robustness to deeper inheritance chains.

Finally, Table 11c evaluates calibration quality as a function of path distance using Brier scores. Gemma-4-31B achieves near-zero scores across both short and long paths, indicating highly calibrated confidence estimates. GPT-5.1, despite strong accuracy, exhibits substantially larger Brier scores, showing that accurate answers do not necessarily correspond to well-calibrated belief states.

Overall, the on-path analysis suggests that models differ not only in raw accuracy, but also in how confidence degrades as reasoning depth increases. Some models maintain stable calibrated beliefs over long inheritance chains, whereas others become progressively less certain as the inferential distance grows.

G.2 Off-Path Scope Sensitivity

We next analyze updates that occur outside the active inheritance path. In tree inheritance, these updates generally belong to sibling branches and therefore should have no effect on the hypothesis. This setting provides a direct test of whether models correctly distinguish structurally relevant from irrelevant evidence.

Table 12a reports correlations between off-path distance and confidence dynamics, together with intrusion rates and qualitative outcome classifications. Models categorized as *aware* become less disturbed by updates as structural distance increases, whereas *confused* models react more strongly to distant off-path evidence. GPT-5.1 shows the lowest intrusion rates and mild distance awareness, while Gemma-4-E4B and Llama-3.1-8B exhibit substantially higher intrusion rates, indicating that they often revise their beliefs in response to causally irrelevant sibling-branch updates.

Table 12b further measures signal discrimination by comparing the magnitude of confidence change on on-path versus off-path turns. A ratio greater than 1 indicates that the model responds more strongly to causally relevant updates than to irrelevant ones. GPT-5.1 consistently shows strong positive discrimination ratios, suggesting that it distinguishes relevant from irrelevant evidence relatively well. In contrast, both Llama models exhibit ratio inversion (< 1), meaning that off-path updates often perturb confidence more strongly than genuinely causal on-path updates.

Discussion

The on-path results establish that longer inference chains genuinely erode confidence in standard instruction-tuned models, an effect confirmed by both non-parametric correlations and mixed-effects regression. The effect is not uniform: Gemma-4-31B shows the strongest degradation ($\beta = -0.060$) yet maintains near-perfect calibration (Brier ≈ 0), whereas GPT-5.1 is poorly calibrated (Brier > 0.49) without showing path-length degradation. These two failure modes are dissociated.

ble: path-length sensitivity and calibration quality are independent properties of the models tested.

The off-path results reveal a structural blindness that compounds the on-path failure. Models do not use graph distance to modulate their response to off-path updates — they treat structurally irrelevant information similarly to causally relevant information. The most striking manifestation is the ratio inversion in Llama models: at Easy difficulty, Llama-3.1-8B responds $0.53\times$ as strongly to on-path updates as to off-path ones, meaning the irrelevant updates are more disruptive than the relevant ones. Combined with Llama-3.1-8B’s intrusion rate of 36.2 % on Easy — more than one in three off-path turns cause an answer change — this indicates that Llama models lack the structural path awareness needed to gate which updates warrant belief revision.

H Confirmation Bias Experiment

Theoretical Background

Confirmation bias — the tendency to search for, interpret, and recall information in a way that confirms one’s prior beliefs — is one of the most robust and widely replicated findings in cognitive psychology (Nickerson, 1998). Its relevance to language model evaluation arises from the multi-turn nature of defeasible reasoning: a model that has committed to a belief at turn t must be willing to revise that belief at turn $t + 1$ if new evidence warrants it. A model exhibiting confirmation bias will systematically fail to revise, treating incongruent evidence as less compelling than its logical weight justifies.

Prior work has documented content effects in LLM reasoning and susceptibility to cognitive biases more broadly (Jones and Steinhardt, 2022), but defeasible belief updating under formally verified ground truth has not previously been studied in this framework. DERELAB is the first benchmark to provide the controlled conditions necessary to measure confirmation bias as a structural property of belief updating rather than a surface-level sensitivity to semantic content.

Operationalisation

Prior belief. The model’s prior belief at turn t is defined as its predicted answer at the immediately preceding `update_answer` or `initial_answer` turn. Critically, we use the *model’s own predicted answer*, not the ground truth, as the prior. Confirmation bias is a property of what the model resists

updating *away from*, regardless of whether that prior belief was correct.

Prior reset. At every hypothesis change turn, the prior belief tracker resets. The new section’s prior is initialized from the ground truth of the initial answer turn for the new hypothesis subject, not from the model’s answer in the preceding section. This prevents cross-subject contamination of the congruence labels.

Congruence conditions. Each `update_answer` turn is assigned exactly one label from the following mutually exclusive taxonomy:

- **C1 — Congruent, live chain.** Prior belief = *yes*; ground truth effect = *strengthening*. The update aligns with the model’s current belief and the chain remains active.
- **C2 — Incongruent, live chain.** Prior belief = *yes*; ground truth effect = *weakening*. The update should defeat the current conclusion. This is the primary confirmation bias signal.
- **C3 — Saturated semantic negative.** Prior belief = *no*; ground truth effect = *no effect*; update type $\in \{\text{neg}, \text{exception}\}$. The chain is already defeated; the update sounds negative but is logically inert. A model incorrectly labelling this as *weakening* is responding to semantic surface form rather than logical scope — a distinct bias we term *semantic override*.
- **C4 — Neutral distractor.** Prior belief = *no*; ground truth effect = *no effect*; update type $\in \{\text{safe_irr}, \text{quantified_gen}, \text{quantified_inst}\}$. The update is logically irrelevant regardless of semantic content.
- **C5 — From unknown prior.** Prior belief = *unknown*; ground truth effect $\in \{\text{strengthening}, \text{weakening}\}$. Directional updates from an indeterminate state. Analysed separately from C1–C4.
- **C0 — Excluded.** All remaining combinations that do not fit C1–C5 unambiguously. Excluded from all analyses; logged with reason.

Primary bias metric. The confirmation bias gap is defined as:

$$\text{BiasGap} = \text{Acc}(\text{C1}) - \text{Acc}(\text{C2}) \quad (1)$$

where $\text{Acc}(\cdot)$ is turn-level accuracy against formally verified ground truth labels. A positive BiasGap indicates the model performs better on congruent than incongruent updates — confirmation bias.

Odds ratio. As a scale-invariant effect size measure, we additionally compute:

$$\text{OR} = \frac{C1_{\text{correct}} \times C2_{\text{wrong}}}{C1_{\text{wrong}} \times C2_{\text{correct}}} \quad (2)$$

$\text{OR} > 1$ indicates confirmation bias; $\text{OR} = 1$ indicates no asymmetry. Bootstrap 95 % confidence intervals are computed at the conversation level (10,000 resamples).

Statistical tests. McNemar’s test is applied to paired C1/C2 turns within the same hypothesis section, with one C1–C2 pair selected per section. Holm–Bonferroni correction is applied across all per-model tests. A significant result with positive BiasGap constitutes evidence of confirmation bias for that model.

Belief update dissociation. For default reasoning, where the belief-update question is also asked, we additionally measure $\text{Acc}_{\text{effect}}(\text{C2})$: accuracy on the *strengthen/weaken/no effect* question specifically on C2 turns. A model where $\text{Acc}_{\text{effect}}(\text{C2})$ is high but BiasGap is also high exhibits the *know-but-don’t-output* pattern: it correctly identifies that the update weakens the hypothesis but fails to update its entailment answer accordingly.

Anchoring analysis. For each hypothesis section in which the ground-truth label changes between the initial and final turn, we compute the anchoring rate:

$$A = \frac{\left| \left\{ s \in \mathcal{S} : \hat{y}_{\text{final}}^{(s)} = \hat{y}_{\text{init}}^{(s)} \neq y_{\text{final}}^{(s)} \right\} \right|}{\left| \left\{ s \in \mathcal{S} : y_{\text{init}}^{(s)} \neq y_{\text{final}}^{(s)} \right\} \right|} \quad (3)$$

Here, $\mathcal{S}$ denotes the set of hypothesis sections, $\hat{y}_{\text{init}}$ and $\hat{y}_{\text{final}}$ are the model’s initial and final predictions, and y_{init} and y_{final} are the corresponding ground-truth labels. A section counts as anchored when the model preserves its initial prediction despite the correct answer changing over the course of the conversation.

A high anchoring rate indicates that the model maintains its initial conclusion, even after a sequence of updates that should have revised it.

I Confirmation Bias: Full Statistical Results

This appendix provides complete statistical tables and figures for the confirmation bias analysis. All metrics follow the operationalisation in Appendix H.

Primary Bias Metrics

Table 13 reports the full bias metrics for all model–reasoning-type cells. All entries except Qwen3-32B inheritance are significant at $p < 0.001$ after Holm–Bonferroni correction. Qwen3-32B inheritance ($p = 0.149$) is the single non-significant result. OR is undefined (—) when C1 errors = 0, making the odds ratio denominator zero.

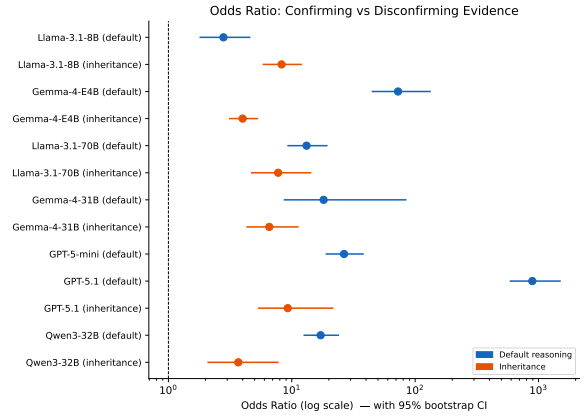


Figure 8: Forest plot of odds ratios (log scale) with bootstrap 95 % confidence intervals per model–reasoning-type cell. $\text{OR} > 1$ indicates confirmation bias. The dashed entry for Qwen3-32B inheritance marks the only non-significant result ($p = 0.149$). GPT-5-mini inheritance is excluded (OR undefined, C1 errors = 0).

Condition Turn Counts

Table 14 reports the number of turns assigned to each congruence condition per model and reasoning type. High C5 counts for Llama-3.1-70B (default: 1,693) and Llama-3.1-8B (default: 875) reflect the evasion bias: these models frequently output *unknown* rather than committing to a direction, reducing the number of turns with a *yes* prior that qualify as C1 or C2.

Anchoring and Semantic Override

Metacognitive Belief Update

Table 16 reports C1 and C2 effect accuracy on the belief-update question (default reasoning only). A negative meta-gap ($\text{C1-effect} < \text{C2-effect}$) indicates the model is better at identifying *weakening*

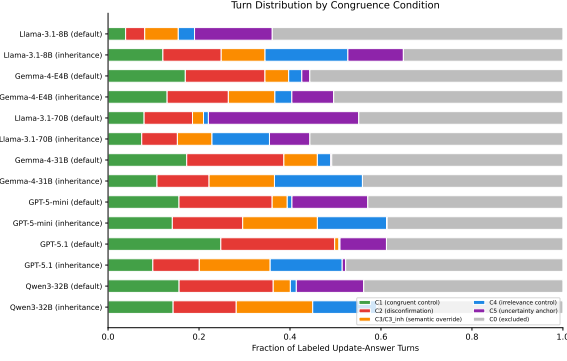


Figure 9: Distribution of congruence conditions per model and reasoning type (fraction of total turns). Large C5 segments for Llama-3.1-70B and Llama-3.1-8B on default reasoning reflect evasion bias. Large C0 segments reflect turns excluded due to prior-belief state not meeting C1–C5 criteria.

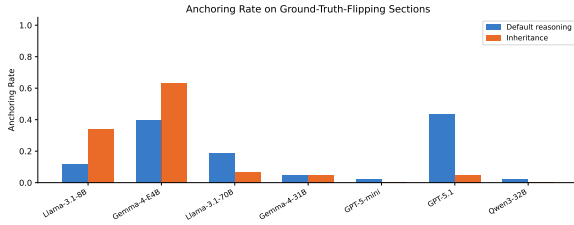


Figure 10: Anchoring rate on ground-truth-flipping sections per model and reasoning type. GPT-5.1 shows the highest default anchoring (0.433); Gemma-4-E4B shows the highest inheritance anchoring (0.635). Qwen3-32B and GPT-5-mini show near-zero anchoring on both types (≤ 0.020).

than *strengthening* — the inverse of what confirmation bias would predict at the metacognitive level. Gemma-4-31B (-0.506) and GPT-5-mini (-0.441) show the strongest inverse metacognitive gap while also showing significant entailment bias, constituting the know-but-don’t-output pattern.

Per-Topology Breakdown

Table 18 reports BiasGap, anchoring rate, and semantic override rate broken down by topology and difficulty. Key observations: GPT-5.1 Hard default anchoring (0.498) is the single highest cell across all models and conditions; Gemma-4-E4B tree Hard anchoring (0.878) is the highest inheritance cell. Llama-3.1-70B Easy default semantic override (0.889) confirms the Llama semantic override finding is not a difficulty artefact.

J Evaluation Prompts

Table 19 lists the system prompts that had been used to evaluate.

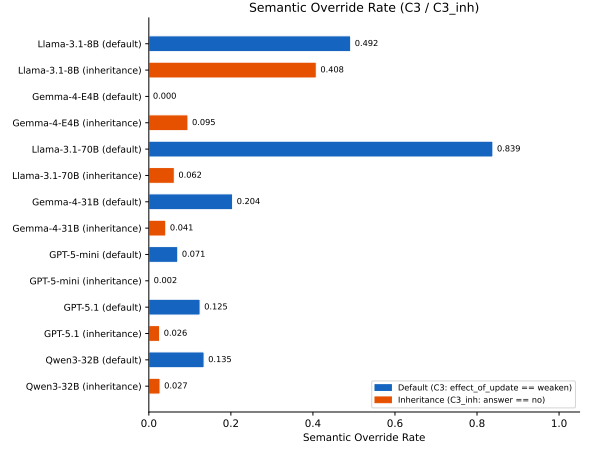


Figure 11: Semantic override rate per model. Default C3 (solid): fraction of turns where model incorrectly labels a logically inert negative update as *weakening* (chain already defeated). Inheritance C3_{inh} (hatched): fraction where model answers *no* on a sibling-branch update that cannot affect the hypothesis subject. Llama-3.1-70B shows an extreme default override (0.839) that does not carry over to inheritance (0.062), indicating sensitivity to surface negation that is specific to the belief-update task format. Gemma-4-E4B shows exactly 0.000 default override.

K Resource Information

Table 17 covers the information about LLMs used in experiments.

For proprietary/closed models (GPT-5.1 and GPT-5-mini), we kept the temperature at 0.1. For open-source models, we used constraint Decoding. We ran 2 A40 GPUs (48*2=96 GB) for approximately 200 hours.

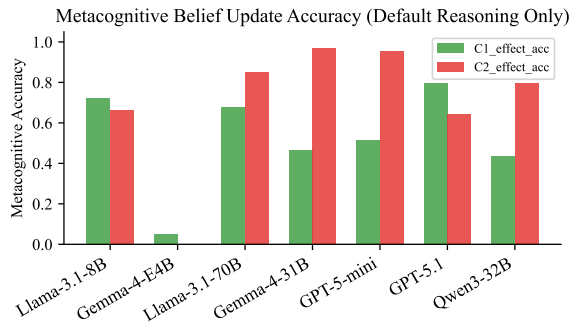


Figure 12: Metacognitive accuracy on the belief-update task (default reasoning only). Bars show C1-effect (blue) and C2-effect (red) accuracy. The dashed overlay shows entailment C2 accuracy for reference. Models where C2-effect is high but entailment C2 accuracy is low (Gemma-4-31B, GPT-5-mini, Qwen3-32B) exhibit the know-but-don’t-output dissociation: correct metacognitive identification of weakening without corresponding entailment revision. Gemma-4-E4B near-zero on both bars indicates failure to use the belief-update label vocabulary.

Model	Diff	n	ρ (conf)	p	ρ (Δ conf)	p	Cohen's d
GPT-5.1	Easy	150	+0.015	0.860	-0.021	0.797	+0.148
GPT-5.1	Hard	294	+0.130	0.026	+0.100	0.087	-0.283
Gemma-4-31B	Easy	150	-0.274	0.001	-0.209	0.010	-0.240
Gemma-4-31B	Hard	294	-0.298	<0.001	-0.298	<0.001	+0.496
Gemma-4-E4B	Easy	150	-0.037	0.651	-0.035	0.670	-0.358
Gemma-4-E4B	Hard	294	+0.072	0.217	+0.024	0.683	-0.133
Llama-3.1-70B	Easy	150	-0.152	0.062	-0.156	0.056	+0.087
Llama-3.1-70B	Hard	294	-0.207	<0.001	-0.237	<0.001	+0.282
Llama-3.1-8B	Easy	150	-0.230	0.005	-0.086	0.295	-0.043
Llama-3.1-8B	Hard	294	-0.255	<0.001	-0.171	0.003	+0.228

(a) Spearman correlations between inference path length and model confidence on on-path turns.

Model	n	$\hat{\beta}$ (distance)	p
GPT-5.1	444	+0.027	0.156
Gemma-4-31B	444	-0.060	<0.001
Gemma-4-E4B	444	-0.011	0.377
Llama-3.1-70B	444	-0.054	<0.001
Llama-3.1-8B	444	-0.047	0.001

(b) Mixed-effects regression coefficient on inference path distance ($\Delta_{\text{gold_conf}} \sim \text{distance} + \text{difficulty} + (1 | \text{conv_id})$). Negative $\hat{\beta}$ indicates each additional hop reduces confidence in the correct label, controlling for conversation-level baseline variance.

Model	Diff	Short path	Long path	Δ
GPT-5.1	Easy	0.4908	0.3585	-0.132
GPT-5.1	Hard	0.6431	0.4883	-0.155
Gemma-4-31B	Easy	0.0000	0.0000	0.000
Gemma-4-31B	Hard	0.0001	0.0002	+0.000
Gemma-4-E4B	Easy	0.0382	0.0077	-0.031
Gemma-4-E4B	Hard	0.0059	0.0058	0.000
Llama-3.1-70B	Easy	0.0259	0.0217	-0.004
Llama-3.1-70B	Hard	0.0527	0.0379	-0.015
Llama-3.1-8B	Easy	0.0586	0.0803	+0.022
Llama-3.1-8B	Hard	0.0357	0.0354	0.000

(c) Brier scores (lower = better calibration) for short (distance ≤ 2) vs long (distance > 2) on-path turns. GPT-5.1's high baseline Brier scores indicate poor calibration despite reasonable accuracy. Gemma-4-31B near-zero scores indicate excellent calibration.

Table 11: Path-confidence analysis on on-path tree-inheritance updates. Detailed interpretations and statistical discussion are provided in Appendix G.

Model	Diff	n	$\rho(\text{conf})$	p	$\rho(\Delta)$	p	Intr.	Outcome
GPT-5.1	Easy	663	+0.092	0.018	-0.086	0.026	0.9 %	(B) Aware
GPT-5.1	Hard	2624	+0.014	0.467	-0.019	0.334	1.2 %	(A) Indf.
Gemma-4-31B	Easy	663	-0.013	0.729	+0.022	0.579	0.3 %	(A) Indf.
Gemma-4-31B	Hard	2624	-0.033	0.089	+0.033	0.089	1.5 %	(A) Indf.
Gemma-4-E4B	Easy	663	+0.015	0.702	+0.171	<0.001	4.2 %	(C) Conf.
Gemma-4-E4B	Hard	2624	+0.104	<0.001	+0.058	0.003	5.6 %	(C) Conf.
Llama-3.1-70B	Easy	663	+0.016	0.674	+0.013	0.742	15.2 %	(A) Indf.
Llama-3.1-70B	Hard	2624	-0.029	0.140	-0.008	0.698	8.1 %	(A) Indf.
Llama-3.1-8B	Easy	663	-0.059	0.132	+0.015	0.693	36.2 %	(C) Conf.
Llama-3.1-8B	Hard	2624	-0.011	0.588	-0.014	0.466	31.9 %	(A) Indf.

(a) Off-path Spearman correlations and outcome classifications.

Model	Diff	On-path $ \Delta $	Off-path $ \Delta $	Ratio	Off-path acc.
GPT-5.1	Easy	0.479	0.319	1.50×	95.9 %
GPT-5.1	Hard	0.576	0.383	1.50×	79.4 %
Gemma-4-31B	Easy	0.027	0.018	1.50×	98.2 %
Gemma-4-31B	Hard	0.117	0.154	0.76×	84.6 %
Gemma-4-E4B	Easy	0.345	0.338	1.02×	45.1 %
Gemma-4-E4B	Hard	0.381	0.382	1.00×	31.1 %
Llama-3.1-70B	Easy	0.148	0.375	0.40×	63.0 %
Llama-3.1-70B	Hard	0.462	0.581	0.79×	32.0 %
Llama-3.1-8B	Easy	0.249	0.466	0.53×	47.8 %
Llama-3.1-8B	Hard	0.330	0.432	0.76×	36.3 %

(b) Signal discrimination between causally relevant and irrelevant updates.

Table 12: Path-confidence analysis on off-path tree-inheritance updates. Full statistical discussion is provided in Appendix G.

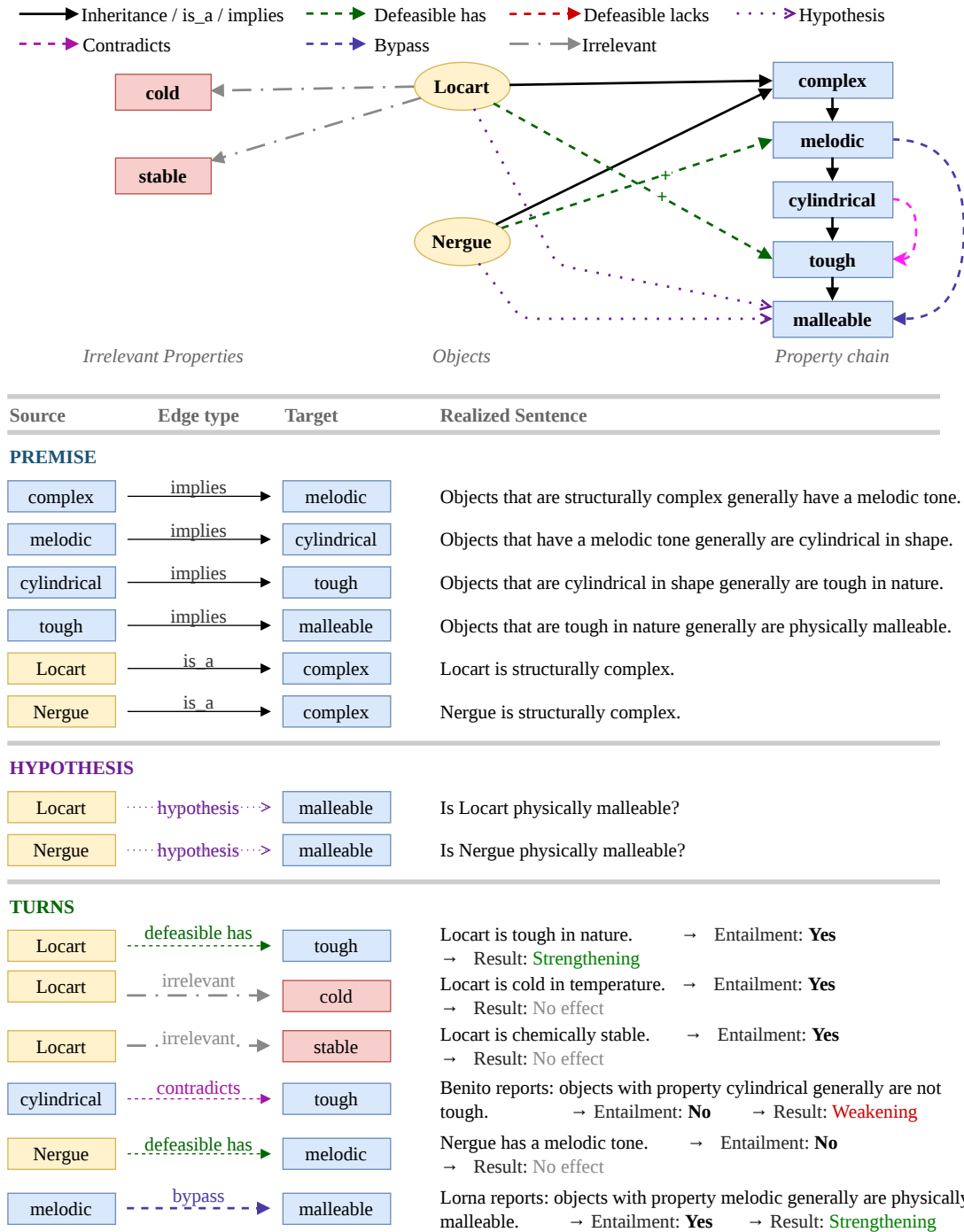
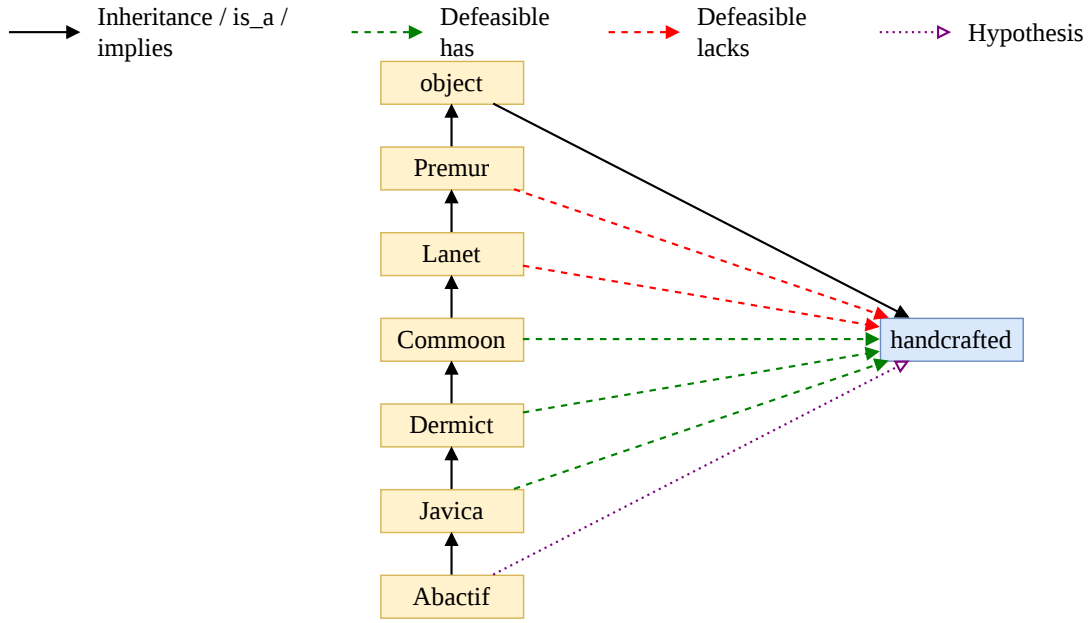
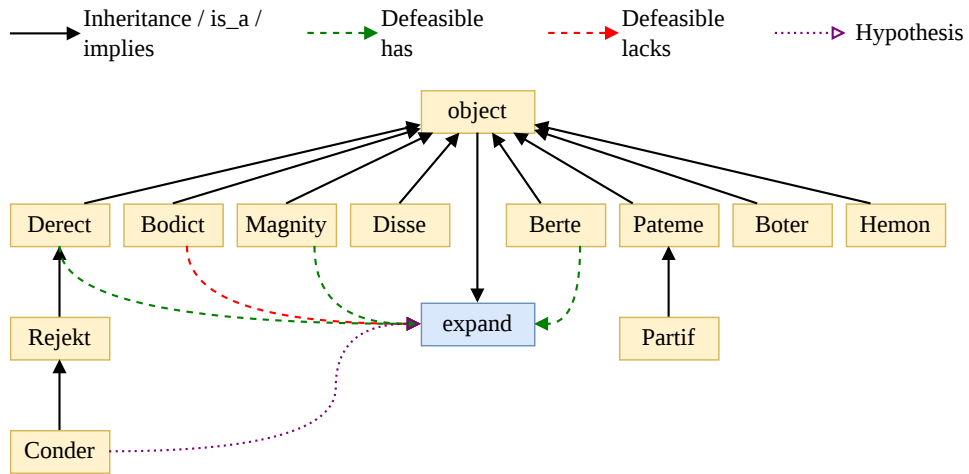


Figure 13: Default Reasoning Example



Source	Edge type	Target	Realized Sentence
PREMISE			
object	implies	handcrafted	Objects are of handcrafted origin.
Premur	is_a	object	Premur is a kind of object.
Lanet	is_a	Premur	Lanet is a kind of Premur.
Commoon	is_a	Lanet	Commoon is a kind of Lanet.
Dermict	is_a	Commoon	Dermict is a kind of Commoon.
Javica	is_a	Dermict	Javica is a kind of Dermict.
Abactif	is_a	Javica	Abactif is a kind of Javica.
HYPOTHESIS			
Abactif	hypothesis	handcrafted	Is Abactif of handcrafted origin?
URNS			
Premur	defeasible lacks	handcrafted	Premur is not of handcrafted origin. → Entailment: No. → Result: Weakening
Lanet	defeasible lacks	handcrafted	Lanet is not of handcrafted origin. → Entailment: No. → Result: Weakening
Commoon	defeasible has	handcrafted	Commoon is of handcrafted origin. → Entailment: Yes. → Result: Strengthening
Dermict	defeasible has	handcrafted	Dermict is of handcrafted origin. → Entailment: Yes. → Result: Strengthening
Javica	defeasible has	handcrafted	Javica is of handcrafted origin. → Entailment: Yes. → Result: Strengthening

Figure 14: Linear Inheritance Example



Source	Edge type	Target	Realized Sentence
PREMISE			
object	implies	expand	Objects can expand.
Derekt	is_a	object	Derekt is a kind of object.
Bodict	is_a	object	Bodict is a kind of object.
Magnity	is_a	object	Magnity is a kind of object.
Disse	is_a	object	Disse is a kind of object.
Berte	is_a	object	Berte is a kind of object.
Pateme	is_a	object	Pateme is a kind of object.
Boter	is_a	object	Boter is a kind of object.
Hemon	is_a	object	Hemon is a kind of object.
Rejekt	is_a	Derekt	Rejekt is a kind of Derekt.
Partif	is_a	Pateme	Partif is a kind of Pateme.
Conder	is_a	Rejekt	Conder is a kind of Rejekt.
HYPOTHESIS			
Conder	hypothesis	expand	can Conder expand?
URNS			
Derekt	defeasible has	expand	Derekt can expand. → Entailment: Yes → Result: Strengthening
Bodict	defeasible lacks	expand	Bodict can not expand. → Entailment: Yes → Result: No effect
Magnity	defeasible has	expand	Magnity can expand. → Entailment: Yes → Result: No effect
Berte	defeasible has	expand	Berte can expand. → Entailment: Yes → Result: No effect

Figure 15: Tree Inheritance Example

Model	Type	C1 acc	C2 acc	Gap	OR	OR lo	OR hi	Holm p
<i>Default reasoning</i>								
GPT-5.1	def	0.970	0.035	0.935	891.2	593.4	1494.0	< 0.001
GPT-5-mini	def	0.871	0.203	0.668	26.5	19.1	37.8	< 0.001
Qwen3-32B	def	0.883	0.305	0.578	17.1	12.6	23.8	< 0.001
Llama-3.1-70B	def	0.697	0.149	0.548	13.2	9.3	19.2	< 0.001
Gemma-4-E4B	def	0.978	0.381	0.597	72.7	45.1	132.2	< 0.001
Llama-3.1-8B	def	0.773	0.549	0.224	2.8	1.8	4.6	< 0.001
Gemma-4-31B	def	0.994	0.907	0.088	18.1	8.7	83.9	< 0.001
<i>Inheritance reasoning</i>								
GPT-5.1	inh	0.958	0.712	0.246	9.3	5.4	21.5	< 0.001
Llama-3.1-8B	inh	0.738	0.255	0.484	8.3	5.9	11.9	< 0.001
Llama-3.1-70B	inh	0.899	0.535	0.364	7.8	4.7	14.2	< 0.001
Gemma-4-31B	inh	0.935	0.686	0.249	6.6	4.3	11.2	< 0.001
Gemma-4-E4B	inh	0.741	0.417	0.324	4.0	3.1	5.3	< 0.001
Qwen3-32B	inh	0.973	0.908	0.065	3.7	2.1	7.7	0.149
GPT-5-mini	inh	1.000	0.997	0.003	—	—	—	—

Table 13: Full primary bias metrics. OR = odds ratio with bootstrap 95 % CI (10,000 resamples, conversation-level). Gap = BiasGap (C1 acc – C2 acc). Entries sorted by Gap descending within each reasoning type. Qwen3-32B inheritance is not significant ($p = 0.149$); GPT-5-mini inheritance OR is undefined because C1 errors = 0.

Model	Type	C1	C2	C3	C4	C5	C0	Paired
<i>Default reasoning</i>								
GPT-5.1	def	1270	1282	48	12	523	1987	553
Gemma-4-31B	def	886	1094	377	150	12	2603	361
GPT-5-mini	def	799	1049	170	53	854	2197	307
Qwen3-32B	def	800	1060	193	67	759	2243	308
Gemma-4-E4B	def	870	897	267	148	90	2850	380
Llama-3.1-70B	def	406	545	124	55	1693	2299	127
Llama-3.1-8B	def	198	215	378	183	875	3273	68
<i>Inheritance reasoning</i>								
GPT-5-mini	inh	927	1015	1077	1000	6	2535	245
GPT-5.1	inh	646	670	1022	1039	50	3133	232
Gemma-4-E4B	inh	852	883	671	245	603	3306	213
Llama-3.1-8B	inh	791	841	632	1195	800	2301	247
Gemma-4-31B	inh	706	751	943	1269	0	2891	243
Llama-3.1-70B	inh	486	514	499	829	583	3649	182
Qwen3-32B	inh	559	542	657	553	1	1595	161

Table 14: Turn counts per congruence condition. C0 = excluded turns (see Appendix H for exclusion reasons). Paired = number of matched C1–C2 section pairs available for McNemar testing. Large C5 counts for Llama models indicate evasion bias: models outputting *unknown* rather than committing.

Model	Type	Flipped	Anchored	Rate	C3 override
<i>Default reasoning</i>					
GPT-5.1	def	402	174	0.433	0.125
Gemma-4-E4B	def	402	159	0.396	0.000
Llama-3.1-70B	def	402	76	0.189	0.839
Llama-3.1-8B	def	402	48	0.119	0.492
Gemma-4-31B	def	402	19	0.047	0.204
GPT-5-mini	def	402	8	0.020	0.071
Qwen3-32B	def	402	8	0.020	0.135
<i>Inheritance reasoning</i>					
Gemma-4-E4B	inh	296	188	0.635	0.095
Llama-3.1-8B	inh	296	100	0.338	0.408
GPT-5.1	inh	296	15	0.051	0.026
Llama-3.1-70B	inh	296	20	0.068	0.062
Gemma-4-31B	inh	296	14	0.047	0.041
Qwen3-32B	inh	220	1	0.005	0.027
GPT-5-mini	inh	296	1	0.003	0.002

Table 15: Anchoring rate on ground-truth-flipping sections and semantic override rate on C3 turns. **Flipped**: sections where ground truth changed from initial to final answer. **Anchored**: flipped sections where model maintained its opening answer despite the ground truth changing. **C3 override**: fraction of C3 turns (default) or C3_{inh} turns (inheritance) where model incorrectly labelled a logically inert update as *weakening* or answered *no* respectively. Note Llama-3.1-70B’s extreme default C3 override (0.839) versus near-zero on inheritance (0.062).

Model	C1-eff	n	C2-eff	n	Gap
Gemma-4-31B	0.464	886	0.970	1094	−0.506
GPT-5-mini	0.513	799	0.954	1049	−0.441
Qwen3-32B	0.435	800	0.797	1060	−0.362
Llama-3.1-70B	0.680	406	0.851	545	−0.172
Llama-3.1-8B	0.722	198	0.665	215	+0.057
Gemma-4-E4B	0.049	870	0.000	897	+0.049
GPT-5.1	0.797	1270	0.644	1282	+0.153

Table 16: Metacognitive accuracy on the belief-update task (default reasoning only). **C1-eff**: fraction of C1 turns correctly labelled *strengthening*. **C2-eff**: fraction of C2 turns correctly labelled *weakening*. **Gap** = C1-eff – C2-eff: negative = model better at recognising weakening than strengthening. Models sorted by gap ascending to surface the dissociation pattern. Gemma-4-E4B near-zero on both tasks indicates failure to use the belief-update label vocabulary at all.

Table 17: Overview of Large Language Models used in experiments, including developers, parameter counts, and access types.

Model	Developer	Parameters	Access
GPT-5.1	OpenAI	Undisclosed	Proprietary
GPT-5-mini	OpenAI	Undisclosed	Proprietary
Gemma-4-31B-it	Google	31B	Open source
Gemma-4-E4B	Google	~4B	Open source
Llama-3.1-70B	Meta	70B	Open source
Llama-3.1-8B	Meta	8B	Open source
Qwen3-32B	Alibaba	32B	Open source

Model	Topology	Diff	C1 acc	C2 acc	Gap	Anch.
<i>Default reasoning</i>						
GPT-5.1	default	easy	0.990	0.032	0.958	0.227
GPT-5.1	default	hard	0.966	0.036	0.931	0.498
GPT-5-mini	default	easy	0.890	0.190	0.701	0.000
GPT-5-mini	default	hard	0.867	0.206	0.660	0.026
Qwen3-32B	default	easy	0.908	0.335	0.573	0.000
Qwen3-32B	default	hard	0.877	0.297	0.579	0.026
Llama-3.1-70B	default	easy	0.828	0.175	0.653	0.052
Llama-3.1-70B	default	hard	0.645	0.138	0.507	0.233
Gemma-4-E4B	default	easy	0.992	0.380	0.612	0.289
Gemma-4-E4B	default	hard	0.976	0.382	0.595	0.430
Llama-3.1-8B	default	easy	0.844	0.525	0.319	0.134
Llama-3.1-8B	default	hard	0.727	0.569	0.158	0.115
Gemma-4-31B	default	easy	1.000	0.804	0.196	0.000
Gemma-4-31B	default	hard	0.993	0.932	0.061	0.062
<i>Linear inheritance</i>						
Llama-3.1-8B	linear	easy	0.932	0.228	0.704	0.256
Llama-3.1-8B	linear	hard	0.644	0.223	0.421	0.409
Gemma-4-E4B	linear	easy	0.882	0.472	0.410	0.385
Gemma-4-E4B	linear	hard	0.675	0.433	0.242	0.580
Llama-3.1-70B	linear	easy	0.973	0.737	0.236	0.051
Llama-3.1-70B	linear	hard	0.843	0.387	0.456	0.068
Gemma-4-31B	linear	easy	1.000	0.822	0.178	0.013
Gemma-4-31B	linear	hard	0.888	0.569	0.319	0.114
GPT-5.1	linear	easy	0.993	0.799	0.195	0.000
GPT-5.1	linear	hard	0.931	0.656	0.275	0.023
Qwen3-32B	linear	easy	1.000	1.000	0.000	0.000
Qwen3-32B	linear	hard	0.946	0.837	0.109	0.000
GPT-5-mini	linear	easy	1.000	1.000	0.000	0.000
GPT-5-mini	linear	hard	1.000	0.996	0.004	0.011
<i>Tree inheritance</i>						
Gemma-4-E4B	tree	easy	0.949	0.400	0.549	0.750
Gemma-4-E4B	tree	hard	0.862	0.157	0.704	0.878
Llama-3.1-8B	tree	easy	0.976	0.542	0.435	0.321
Llama-3.1-8B	tree	hard	0.922	0.393	0.528	0.351
Llama-3.1-70B	tree	hard	0.871	0.447	0.424	0.122
Llama-3.1-70B	tree	easy	1.000	0.826	0.174	0.018
GPT-5.1	tree	hard	0.968	0.653	0.315	0.176
GPT-5.1	tree	easy	1.000	0.921	0.079	0.000
Gemma-4-31B	tree	hard	0.988	0.807	0.180	0.041
Gemma-4-31B	tree	easy	1.000	0.937	0.064	0.000
Qwen3-32B	tree	hard	1.000	0.887	0.113	0.000
Qwen3-32B	tree	easy	1.000	1.000	0.000	0.018
GPT-5-mini	tree	hard	1.000	1.000	0.000	0.000
GPT-5-mini	tree	easy	1.000	1.000	0.000	0.000

Table 18: BiasGap and anchoring rate by topology and difficulty. Rows sorted by BiasGap descending within each topology group. The GPT-5.1 Hard default row (Gap=0.931, Anch.=0.498) and the Gemma-4-E4B tree Hard row (Gap=0.704, Anch.=0.878) are the most extreme cells across all conditions.

Prompt Title/Name	Prompt
System Prompt A: Inheritance Defeasible Reasoning	You are an expert in defeasible reasoning. You will be given a category hierarchy where entities belong to categories, and categories can belong to other categories. A property that applies to a category generally applies to all entities and subcategories within it, unless explicitly stated otherwise for a specific entity or subcategory. All previously given information remains active throughout the conversation. You will then receive a series of information updates one at a time. After each update you will be asked whether all the information so far supports the hypothesis. Answer yes if the hierarchy and facts support the hypothesis and nothing blocks inheritance for the specific entity in question, no if inheritance is explicitly blocked for that entity, or unknown if you cannot determine either way. Output only the answer word and nothing else. Do not explain, justify, or add any additional text.
System Prompt B: Defeasible Reasoning with Effect Turns	You are an expert in defeasible reasoning. You will be given a set of general rules and specific facts about objects. The rules are defaults: they hold generally unless specific information contradicts them for a particular object. All previously given information remains active unless explicitly overridden. You will then receive a series of information updates one at a time. After each update you will be asked one of two questions: Support: Does the information support the hypothesis? Answer yes, no, or unknown. Effect: Does the update strengthen, weaken, or have no effect on the support for the hypothesis? When information comes from multiple sources, rely on the most reliable source when sources contradict each other. If a less reliable source makes a claim and a more reliable source makes a different but non-contradicting claim, both claims remain active independently. A more reliable source speaking does not undo what a less reliable source established unless they are making opposite claims about exactly the same fact. Output only the answer word and nothing else. Do not explain, justify, or add any additional text.

Table 19: System Prompts for Evaluation